

**M A S A R Y K  
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**A Comparative Study of Automated  
and Manual Creation of Test  
Management Plans and Test Cases**

Master's Thesis

ABDUL BASIT

Brno, Fall 2025

**MASARYK  
UNIVERSITY**

FACULTY OF INFORMATICS

**A Comparative Study of Automated  
and Manual Creation of Test  
Management Plans and Test Cases**

Master's Thesis

ABDUL BASIT

Advisor: Prof. Ing. Tomáš Vojnar, Ph.D.

Faculty of Informatics

Brno, Fall 2025



## Declaration

I hereby declare that this thesis is my original work, which I have written independently. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used the following AI tools: Claude AI for literature review synthesis, grammar checks, improving writing style, and as a research subject for experimental evaluation; ChatGPT, Microsoft Copilot, DeepSeek, and Grok as research subjects for the experimental assessment of test case generation capabilities; Google Gemini for data analysis and results interpretation. I confirm that I have used these tools while strictly following academic integrity guidelines. I have checked the content with reliable sources and my own experimental data, and I take full responsibility for the results.

Abdul Basit

**Advisor:** Prof. Ing. Tomáš Vojnar, Ph.D.

## Acknowledgements

I want to express my sincere gratitude to my supervisor, Prof. Ing. Tomáš Vojnar, Ph.D., for his expert guidance, insightful feedback, and ongoing support throughout this research project. His depth of knowledge in software testing, combined with his constructive suggestions, played a crucial role in shaping the direction and quality of this work.

I am also thankful for the valuable contributions of my consultant, Ing. Patrik Procházka, which significantly enhanced the experimental methodology of this study.

Finally, I would like to extend my heartfelt thanks to my family and friends for their unwavering support, motivation, and encouragement. Their belief in my abilities helped me persevere through the most challenging stages of this journey.

## Abstract

Software testing is a critical yet resource-intensive activity in the software development lifecycle, motivating decades of research into automated testing techniques. Traditional automated testing approaches, such as random testing, symbolic execution, search-based testing, and model-based testing, primarily derive test cases from source code or formal models, focusing on structural coverage metrics or the detection of low-level faults. In contrast, manual testing remains essential for validating high-level functional requirements and business logic specified outside the code.

Recent advances in large language models (LLMs) have introduced new possibilities for automating test artefact creation directly from natural-language requirements. This thesis examines the extent to which LLMs can support requirements-driven test case generation and compares their outputs to those produced by professional human testers. Five prominent LLMs, ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude, are evaluated using a two-phase experimental design. The first phase assesses the models' ability to extract functional and non-functional requirements from Software Requirements Specification (SRS) documents. In contrast, the second phase evaluates the quality of the generated test cases through a progressive comparison that culminates in a statistically validated human-versus-AI evaluation.

The results indicate that the best-performing model achieves high accuracy, strong requirements traceability, and substantially reduced test case generation time compared to human testers. However, the models exhibit limitations in completeness, contextual understanding, and risk-based prioritisation. These findings suggest that LLM-based test generation does not replace traditional automated testing techniques nor human expertise, but rather complements them by operating at the requirements level. The thesis concludes that hybrid testing workflows, combining classical automation, LLM-assisted generation, and human judgment, offer the most effective approach for achieving high-quality and efficient software testing.

## **Keywords**

LLM, Software Engineering, Testing

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>1</b>  |
| <b>2</b> | <b>Background and Related Work</b>                      | <b>3</b>  |
| 2.1      | Software Testing and Requirements Engineering . . . . . | 3         |
| 2.2      | Large Language Models . . . . .                         | 3         |
| 2.3      | AI in Software Engineering . . . . .                    | 4         |
| 2.4      | Related Work . . . . .                                  | 5         |
| <b>3</b> | <b>Research Methodology</b>                             | <b>7</b>  |
| 3.1      | Research Design Overview . . . . .                      | 7         |
| 3.1.1    | Two-Phase Progressive Structure . . . . .               | 7         |
| 3.1.2    | Model Selection and Configuration . . . . .             | 8         |
| 3.1.3    | Experimental Artifact . . . . .                         | 8         |
| 3.2      | Requirements Extraction Evaluation . . . . .            | 9         |
| 3.2.1    | Experimental Procedure . . . . .                        | 9         |
| 3.2.2    | Evaluation Metrics . . . . .                            | 9         |
| 3.3      | Test Case Generation Evaluation . . . . .               | 10        |
| 3.3.1    | Stage 1: Exploratory Five-Model Assessment . .          | 10        |
| 3.3.2    | Stage 2: Systematic Three-Model Comparison .            | 10        |
| 3.3.3    | Stage 3: Comprehensive Human vs AI Evaluation           | 11        |
| 3.4      | Dual-Perspective Analytical Approach . . . . .          | 11        |
| 3.4.1    | Researcher’s Primary Analysis . . . . .                 | 11        |
| 3.4.2    | LLM-Assisted Verification . . . . .                     | 12        |
| 3.4.3    | Synthesis Process . . . . .                             | 12        |
| 3.5      | Statistical Analysis Methods . . . . .                  | 12        |
| 3.5.1    | Inter-Rater Reliability . . . . .                       | 13        |
| 3.5.2    | Significance and Effect Size . . . . .                  | 13        |
| 3.6      | Methodological Limitations . . . . .                    | 13        |
| <b>4</b> | <b>Requirements Extraction Analysis</b>                 | <b>14</b> |
| 4.1      | Experimental Setup . . . . .                            | 14        |
| 4.2      | Overall Performance Summary . . . . .                   | 15        |
| 4.3      | Key Findings by Metric . . . . .                        | 15        |
| 4.3.1    | Coverage . . . . .                                      | 15        |
| 4.3.2    | Clarity . . . . .                                       | 16        |
| 4.3.3    | Traceability . . . . .                                  | 16        |



|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 4.3.4    | Completeness . . . . .                                          | 16        |
| 4.3.5    | Redundancy . . . . .                                            | 17        |
| 4.3.6    | Hallucination . . . . .                                         | 17        |
| 4.4      | Critical Observations . . . . .                                 | 17        |
| 4.5      | Visual Processing Limitations . . . . .                         | 18        |
| 4.6      | Model Selection for Phase 2 . . . . .                           | 19        |
| <b>5</b> | <b>Test Case Generation Evaluation</b>                          | <b>20</b> |
| 5.1      | Exploratory Five-Model Assessment . . . . .                     | 20        |
| 5.1.1    | Experimental Approach . . . . .                                 | 20        |
| 5.1.2    | Aggregated Performance Analysis . . . . .                       | 20        |
| 5.1.3    | Key Model Characteristics . . . . .                             | 21        |
| 5.1.4    | Conclusions and Model Selection . . . . .                       | 22        |
| 5.2      | Systematic Three-Model Comparison . . . . .                     | 22        |
| 5.2.1    | Experimental Framework . . . . .                                | 23        |
| 5.2.2    | Initial Three-Model Comparison . . . . .                        | 23        |
| 5.3      | Focused Two-Model Evaluation . . . . .                          | 24        |
| 5.4      | Comparative Analysis . . . . .                                  | 25        |
| 5.4.1    | Overall Assessment and Claude Selection . . . . .               | 25        |
| <b>6</b> | <b>Comprehensive Human vs. AI Evaluation</b>                    | <b>27</b> |
| 6.1      | Experimental Design . . . . .                                   | 27        |
| 6.1.1    | Evaluation Framework . . . . .                                  | 27        |
| 6.1.2    | Dual-Perspective Verification and Statistical Methods . . . . . | 28        |
| 6.1.3    | Experimental Progression . . . . .                              | 28        |
| 6.2      | Overall Quantitative Results . . . . .                          | 29        |
| 6.3      | Key Findings by Dimension . . . . .                             | 29        |
| 6.3.1    | Accuracy and Hallucination Control . . . . .                    | 29        |
| 6.3.2    | Completeness and Contextual Understanding . . . . .             | 30        |
| 6.3.3    | Requirements Traceability . . . . .                             | 30        |
| 6.3.4    | Oracle Quality and Measurability . . . . .                      | 31        |
| 6.3.5    | Risk Prioritization and Business Judgment . . . . .             | 31        |
| 6.3.6    | Test Suite Structure and Redundancy . . . . .                   | 31        |
| 6.3.7    | Efficiency and Productivity . . . . .                           | 32        |
| 6.4      | Qualitative Observations and Patterns . . . . .                 | 32        |
| 6.4.1    | Recommended Hybrid Workflow . . . . .                           | 33        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>7</b> | <b>Challenges and Limitations</b>                            | <b>35</b> |
| 7.1      | AI-Specific Technical Challenges . . . . .                   | 35        |
| 7.1.1    | Hallucination and Over-Inference . . . . .                   | 35        |
| 7.1.2    | Contextual Understanding Limitations . . . . .               | 36        |
| 7.1.3    | Risk Prioritization Deficiencies . . . . .                   | 37        |
| 7.1.4    | Environmental and Operational Unawareness . . . . .          | 37        |
| 7.2      | Methodological Limitations . . . . .                         | 38        |
| 7.2.1    | Single SRS Document Constraint . . . . .                     | 38        |
| 7.2.2    | Model Version Temporality . . . . .                          | 39        |
| 7.2.3    | Evaluation Subjectivity . . . . .                            | 39        |
| 7.2.4    | Training Data Quality and Adversarial Risks . . . . .        | 40        |
| 7.2.5    | Human Baseline Variability . . . . .                         | 41        |
| 7.3      | Practical Implementation Challenges . . . . .                | 41        |
| 7.3.1    | Team Adoption and Cultural Factors . . . . .                 | 42        |
| 7.3.2    | Cost-Benefit Ambiguity . . . . .                             | 42        |
| 7.3.3    | Organizational Readiness Prerequisites . . . . .             | 43        |
| 7.4      | Broader Implications and Concerns . . . . .                  | 44        |
| 7.4.1    | Professional Skill Evolution . . . . .                       | 44        |
| 7.4.2    | Quality Assurance Philosophy . . . . .                       | 44        |
| 7.4.3    | Accountability and Liability . . . . .                       | 44        |
| <b>8</b> | <b>Conclusion</b>                                            | <b>46</b> |
| 8.1      | Research Summary . . . . .                                   | 46        |
| 8.2      | Answers to Research Questions . . . . .                      | 47        |
| 8.3      | Research Contributions . . . . .                             | 51        |
| 8.4      | Practical Recommendations . . . . .                          | 52        |
| 8.4.1    | For Organisations Considering AI Test Generation . . . . .   | 52        |
| 8.4.2    | For QA Professionals . . . . .                               | 53        |
| 8.4.3    | For Tool Developers . . . . .                                | 53        |
| 8.4.4    | For Researchers . . . . .                                    | 54        |
| 8.4.5    | Custom Model Development and Fine-Tuning Strategy . . . . .  | 54        |
| 8.4.6    | Practical Implementation Approach . . . . .                  | 56        |
| 8.5      | Further Research Directions . . . . .                        | 58        |
| 8.5.1    | Improvement of Non-Functional Requirement Handling . . . . . | 58        |
| 8.6      | Future Work . . . . .                                        | 62        |
| 8.7      | Final Remarks . . . . .                                      | 64        |



# 1 Introduction

Software testing is an essential component of quality assurance within the software development lifecycle. However, it is also one of the most resource-intensive and time-consuming activities, with test case creation and maintenance representing a substantial share of overall development costs. Manual development of test cases not only imposes a significant financial burden but also introduces the risk of human error, particularly as software systems become increasingly complex and development cycles shorten under competitive pressures.

To mitigate these challenges, automated testing techniques have been studied and applied for decades. Traditional automated testing approaches such as random testing, symbolic execution, search-based testing, and model-based testing are typically derived from source code or formal models. These techniques primarily aim to detect low-level faults (e.g., null pointer dereferences, assertion violations) and to maximise structural coverage under various coverage metrics. They have proven highly effective for improving implementation-level quality; however, they generally operate at a low level of abstraction and require access to code or precise models. As a result, they are less suited to validating whether a system fulfils high-level functional requirements and business intent, which are often specified outside the code.

Consequently, manual testing remains indispensable, particularly for system-level and acceptance testing. Human testers excel at interpreting Software Requirements Specification (SRS) documents, reasoning about domain context, identifying implicit requirements, and prioritising tests based on business risk. Despite these strengths, manual test case creation is labour-intensive, difficult to scale, and costly, motivating continued research into complementary approaches that can support testers at higher levels of abstraction.

Recent advances in large language models (LLMs) have presented new opportunities for automating tasks that human experts have traditionally performed. Trained on large corpora of natural language and code, LLMs demonstrate strong capabilities in understanding textual specifications and generating structured artefacts. In the context of software testing, this has led to growing interest in using LLMs

to generate test cases directly from natural-language requirements documents. Unlike traditional automated testing approaches, which are predominantly code-centric, LLM-based test generation operates at the level of high-level requirements, aiming to translate externally specified functional and non-functional requirements into test artefacts.

Despite their potential, the practical effectiveness of LLM-generated test cases remains uncertain. While these models have demonstrated promising results in controlled settings, real-world requirements are frequently incomplete, ambiguous, or domain-specific, posing challenges for automated interpretation. Moreover, although prior work has explored automated and AI-assisted test generation, systematic empirical comparisons between AI-generated and human-authored test cases, particularly at the requirements level, remain limited.

This thesis investigates the strengths and weaknesses of five prominent LLMs: ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude in generating test cases from SRS documents. The study employs a two-phase experimental approach. Phase 1 evaluates each model's ability to extract functional and non-functional requirements from requirement specifications, establishing baseline capability. Phase 2 assesses the quality of generated test cases through a progressive evaluation that culminates in a statistically validated comparison between the best-performing AI model and professional human testers.

The contribution of this work is not to replace traditional automated testing techniques nor human testers, but to examine a complementary, requirements-driven form of automation enabled by LLMs. The results demonstrate that LLMs can significantly improve efficiency and traceability in test case generation, while still exhibiting limitations in contextual understanding, completeness, and risk-based prioritisation. These findings support hybrid testing workflows in which classical automation, LLM-assisted test generation, and human judgement are combined to achieve effective, scalable, and high-quality software testing.

## **2 Background and Related Work**

This chapter establishes the theoretical foundation for investigating AI-assisted test case generation. It synthesizes key concepts from software testing, requirements engineering, and large language models, before reviewing related work that motivates the current study.

### **2.1 Software Testing and Requirements Engineering**

Software testing constitutes a critical yet resource-intensive phase of the development lifecycle, encompassing unit, integration, system, and acceptance testing stages. Test cases form the structural foundation of this process, comprising identifiers, preconditions, test steps, expected outcomes, and pass/fail criteria. Professional standards such as IEEE 829 ensure consistency and maintainability across development phases. Effective test management extends beyond individual test cases to encompass strategic planning, resource allocation, and requirements traceability matrices that link each test to specific requirements, preventing gaps and redundancy.

The quality and structure of Software Requirements Specification (SRS) documents directly influence testing effectiveness. SRS documents formalise system functionality, constraints, and quality attributes, serving as blueprints for both implementation and validation. Requirements are categorised as functional (defining system behaviour) and non-functional (specifying performance, security, usability, and reliability constraints). Testability depends critically on requirement characteristics: they must be measurable in quantifiable terms, deterministic in defining unambiguous behaviours, and independent to allow isolated verification. Poorly articulated requirements propagate ambiguity into test cases, reducing effectiveness regardless of whether generation is manual or automated.

### **2.2 Large Language Models**

Large language models represent neural architectures trained on extensive text corpora to process and generate natural language. Modern

implementations predominantly employ transformer architectures introduced by Vaswani et al., utilising attention mechanisms for efficient sequential data processing. These models exhibit capabilities spanning comprehension, generation, summarisation, and translation tasks. Recent developments have extended base models through instruction tuning and reinforcement learning from human feedback, enhancing alignment with user intent and task-specific performance.

This research evaluates five prominent models: GPT-4 (ChatGPT), Microsoft Copilot (GPT-4 with retrieval augmentation), DeepSeek (an open-source architecture), Grok (real-time information integration), and Claude (a constitutional AI approach emphasising safety). Each represents distinct development philosophies and architectural choices, providing diverse perspectives on requirements interpretation and test generation capabilities.

Despite impressive performance, LLMs exhibit inherent limitations relevant to software testing applications. Hallucination, which generates plausible but factually incorrect outputs, poses risks when models infer unstated requirements or fabricate system behaviours. Contextual understanding remains bounded; models struggle with implicit domain knowledge, organisational conventions, and unspoken assumptions that human testers naturally incorporate. Training data cutoffs impose temporal limitations, and models often lack a genuine understanding of causality, risk prioritisation, and real-world operational constraints.

## **2.3 AI in Software Engineering**

AI applications in software engineering have expanded beyond traditional program analysis to encompass code generation, documentation synthesis, and requirements processing. GitHub Copilot and similar tools demonstrate practical utility in code completion and boilerplate generation. In the testing domain, AI techniques span multiple paradigms. Search-based approaches optimise test inputs using genetic algorithms or simulated annealing. Reinforcement learning trains agents to explore state spaces systematically. Neural program synthesis generates tests from input-output specifications, though primarily at the unit level.

Recent work has begun exploring LLMs for requirements-level test generation. However, most studies focus on demonstrating technical feasibility rather than comprehensive quality evaluation. Existing approaches often require significant manual intervention, formal model construction, or operate within narrow domains. The gap between technical capability demonstrations and practical effectiveness in realistic software testing scenarios motivates the current investigation.

## 2.4 Related Work

Several research directions have explored automated test generation, though comprehensive comparisons between AI-generated and human-written test cases remain limited.

Traditional automated testing techniques, including random testing, symbolic execution, search-based testing, and model-based testing, derive test cases from code or formal models, targeting structural coverage and low-level fault detection. While highly effective for implementation-level quality assurance, these methods operate at abstractions distant from high-level requirements validation.

Natural language processing approaches have attempted requirements-driven test generation with varying success. Hayes et al. (2006) employed NLP and rule-based transformations, demonstrating feasibility but requiring extensive manual rule creation with limited linguistic flexibility. Allala et al. (2022) combined model-driven engineering with NLP for abstract test case generation, necessitating formal model construction that imposed high upfront investment. Wang (2020) showed promise with use case specifications but struggled with less structured requirement formats.

Recent LLM applications have yielded encouraging preliminary results. Halдар et al. (2024) explored the use of ChatGPT for test planning and generation, noting promising outcomes that are tempered by concerns about hallucination and the need for human oversight. However, their evaluation lacked a detailed comparison with human-generated baselines. Arora et al. (2024) demonstrated that retrieval-augmented LLMs improve test scenario quality through context enrichment, though evaluation centred on scenario outlines rather than complete test cases. Chenail-Larcher et al. (2025) found that hy-



brid approaches combining traditional techniques with LLMs outperformed either method independently in IoT contexts, suggesting complementary strengths.

Despite these advances, systematic empirical comparisons evaluating AI-generated test cases against human-written equivalents across comprehensive quality dimensions remain scarce. Existing research predominantly demonstrates technical feasibility or evaluates isolated attributes, such as coverage, leaving gaps in understanding practical effectiveness, maintainability, completeness, and real-world applicability. This thesis addresses these gaps through a rigorous, multi-phase comparison of five LLMs against professional human testers, providing empirical evidence to inform the practical integration of AI-assisted test generation.

## 3 Research Methodology

This chapter presents the systematic approach employed to investigate generative AI capabilities in test case generation. The research uses a multi-phase experimental design that integrates quantitative measurements and qualitative analysis, characterised by a dual-perspective evaluation combining independent researcher analysis with LLM-assisted verification to ensure a comprehensive assessment while maintaining analytical rigour.

### 3.1 Research Design Overview

The investigation employs a progressive experimental methodology that systematically evaluates five large language models while building toward comprehensive comparison with human-generated test cases.

#### 3.1.1 Two-Phase Progressive Structure

This research employs a two-phase progressive evaluation framework designed to systematically assess and compare the capabilities of five generative AI models in software testing tasks.

**Phase 1: Requirements Extraction Analysis** establishes foundational capability by evaluating how effectively each model interprets software requirements documentation. Seven experiments assess extraction of functional and non-functional requirements from various document formats, including formal SRS documents, user stories, and use case diagrams. Performance determines advancement to test case generation evaluation.

**Phase 2: Test Case Generation and Evaluation** employs a three-stage progressive methodology:

- **Stage 1: Exploratory Five-Model Assessment** conducts initial evaluation of all five models' test case generation capabilities, establishing preliminary performance rankings.
- **Stage 2: Systematic Three-Model Comparison** focuses on the top three performers (ChatGPT, Microsoft Copilot, Claude),

applying rigorous evaluation criteria to identify the strongest model while eliminating underperformers.

- **Stage 3: Comprehensive Human vs AI Evaluation** performs detailed comparison between the best-performing AI model and professional human testers across 15 controlled experiments, employing double-blind validation and statistical significance testing.

This progressive elimination strategy balances thoroughness with efficiency, allowing deep investigation of the most promising models while maintaining experimental rigor throughout.

### 3.1.2 Model Selection and Configuration

Five prominent large language models were evaluated: ChatGPT (OpenAI), Microsoft Copilot, DeepSeek, Grok, and Claude (Anthropic). Selection criteria included current availability, documented capabilities in generating structured output, and representation of diverse development approaches. Claude, the most extensively evaluated model, operated with temperature fixed at 0.0 and top-p at 1.0 to maximise output determinism and reproducibility. Fixed random seeds ensured consistency across experimental trials, enabling reliable inter-model comparisons.

### 3.1.3 Experimental Artifact

The CORADS (Car On Roads Advertisement) Software Requirements Specification serves as the primary experimental artefact. This document outlines a location-based mobile advertising platform that supports multiple user roles, provides real-time tracking, facilitates payment processing, and enables campaign management functionality. CORADS proved particularly suitable due to its adherence to professional quality and IEEE standards, comprehensive coverage of both functional and non-functional requirements, realistic domain complexity, and sufficient completeness to support meaningful test generation, while also containing inherent ambiguities that reveal model limitations.

## 3.2 Requirements Extraction Evaluation

Phase 1 establishes fundamental capabilities by evaluating requirement extraction and categorization effectiveness. Accurate requirement comprehension constitutes a prerequisite for meaningful test case generation.

Each model received identical prompts instructing comprehensive extraction with proper categorization and structured output formatting. The extraction task demands linguistic understanding and domain knowledge, as requirements appear distributed throughout specifications intermixed with contextual information, examples, and design discussions.

### 3.2.1 Experimental Procedure

Each model received identical prompts instructing comprehensive extraction with proper categorization and structured output formatting. The extraction task demands both linguistic understanding and domain knowledge, as requirements appear distributed throughout specifications intermixed with contextual information, examples, and design discussions.

### 3.2.2 Evaluation Metrics

The six main metrics used to evaluate the quality of requirement extraction are:

**Coverage** measures the percentage of actual requirements that are successfully identified against a manually derived ground truth reference set.

**Clarity** evaluates the readability and precision of extracted formulations using a qualitative rubric that considers linguistic quality, technical accuracy, and stakeholder accessibility.

**Traceability** examines whether extractions maintain clear relationships to source material through unique identifiers, section references, and hierarchical organisation.

**Completeness** assesses inclusion of all requirement types, including business rules, legal requirements, testing considerations, and quality attributes.

**Redundancy** identifies duplicate or unnecessarily overlapping requirements. Lower scores indicate better performance.

**Hallucination** detection identifies fabricated requirements lacking a foundation in source documentation, which is the most critical concern, as hallucinated requirements could mislead development efforts.

### 3.3 Test Case Generation Evaluation

Phase 2 systematically evaluates test case generation capabilities through two progressive stages with models selected from Phase 1.

#### 3.3.1 Stage 1: Exploratory Five-Model Assessment

Stage 1 conducts an initial exploratory assessment through three experiments, where each model receives identical SRS excerpts and prompts to generate comprehensive test case suites. Evaluation emphasises qualitative assessment of test case structure and organisation, requirements coverage breadth, traceability to source requirements, professional formatting and clarity, and practical usability.

#### 3.3.2 Stage 2: Systematic Three-Model Comparison

Stage 2 performs rigorous systematic comparison of top three performers through four controlled experiments. Each model's output receives 100-point scale scoring across seven dimensions: accuracy (alignment with SRS requirements), completeness (inclusion of required test case fields), test coverage (functional and non-functional requirements), efficiency (time and effort required), quality (detail, specificity, maintainability), bug detection capability, and usability (automation readiness).

Comparative analysis revealed Claude and Microsoft Copilot as strongest performers, with Claude demonstrating superior completeness and non-functional requirements coverage. ChatGPT showed limitations in depth and breadth, particularly for complex scenarios. Claude was selected for comprehensive human comparison based on these results.

### 3.3.3 Stage 3: Comprehensive Human vs AI Evaluation

Stage 3 conducts rigorous 15-experiment comparison between Claude and professional human testers, employing expanded evaluation framework measuring accuracy and hallucination control, completeness and contextual understanding, requirements traceability (RTM coverage), oracle quality and measurability, risk prioritization and business judgment, test suite structure and redundancy, and efficiency and productivity.

**Progressive Prompt Refinement Strategy:** Prompt engineering underwent gradual refinement across iterations to improve AI model performance. This process involved optimizing prompt structure, instruction clarity, and AI role assignments, which directly influenced output quality.

**Statistical Validation:** All quantitative comparisons employ appropriate statistical tests, paired t-tests for continuous metrics, Wilcoxon signed-rank tests for non-parametric data, Cohen's d for effect size calculation, and Cohen's kappa for inter-rater reliability.

## 3.4 Dual-Perspective Analytical Approach

A distinguishing methodological feature involves dual-perspective analysis combining researcher's independent evaluation with LLM-assisted verification.

### 3.4.1 Researcher's Primary Analysis

For each experiment, comprehensive analysis leveraged testing domain expertise and quality assessment knowledge, utilising structured rubrics while incorporating qualitative judgment on appropriateness, practical usability, and alignment with professional standards. Human-led analysis proved crucial because software testing requires contextual understanding, recognising when seemingly correct test cases miss critical scenarios, identifying subtle quality differences, and evaluating practical utility in real-world workflows. The researcher systematically examined test cases, uncovering model behaviour patterns, characteristic hallucination manifestations, and systematic differences between AI-generated and human-generated strategies.

### 3.4.2 LLM-Assisted Verification

Following primary analysis, a large language model received prompts requesting independent evaluation of identical test case sets. This secondary analysis served multiple purposes: Bias Mitigation, researcher expectations and potential biases are countered by LLM analysis operating under different assumptions that may identify overlooked aspects; Consistency Verification, substantial agreement between AI and human assessments increases confidence in conclusions, while disagreements prompt re-examination; Alternative Perspective, LLMs may recognize patterns or relationships less apparent to human reviewers through statistical analysis of training data; Detailed Documentation, LLM analysis generates extensive, structured documentation enriching the research record. Verification prompts maintained independence by withholding the researcher's conclusions, instead providing test cases, evaluation criteria definitions, and requesting a systematic assessment with justification.

### 3.4.3 Synthesis Process

The dual-perspective approach culminated in a synthesis where areas of strong agreement were accepted without further scrutiny, while disagreements prompted a deeper investigation of specific test cases or evaluation dimensions. The researcher's evaluation took precedence for final decisions, as human judgment constitutes the authoritative factor in software testing contexts. However, disagreements often highlighted legitimate ambiguities, prompting more careful consideration and improving overall analysis quality. This methodology strengthens credibility by recognising both human expertise and AI analytical contributions, positioning AI as an analytical aid rather than an autonomous decision-maker.

## 3.5 Statistical Analysis Methods

Statistical analysis establishes confidence in quantitative findings and determines whether observed differences achieve significance.

### **3.5.1 Inter-Rater Reliability**

Cohen's kappa coefficient measures the agreement between evaluators for subjective assessments that require judgment, such as quality ratings (on a 1-5 scale) and priority classifications (High/Medium/Low). The research targeted a kappa value of 0.70, indicating substantial agreement. When preliminary assessments yielded lower kappa values, evaluators discussed criteria, clarified ambiguities, and reassessed cases until adequate agreement was reached, ensuring consistent and reliable subjective judgments.

### **3.5.2 Significance and Effect Size**

Paired t-tests and Wilcoxon signed-rank tests assessed whether performance differences achieved statistical significance ( $p < 0.05$  threshold). Cohen's d quantifies the magnitude of the difference following conventional guidelines: 0.2 (minor effects), 0.5 (medium effects), and 0.8+ (significant effects). The research reports both statistical significance and effect size for key comparisons, providing a comprehensive understanding of performance differences.

## **3.6 Methodological Limitations**

The research acknowledges inherent limitations. A single SRS document focus ensures necessary experimental control, but it limits generalizability across different domains. Model version temporality reflects capabilities at the time of investigation; newer releases may exhibit different performance characteristics. Despite dual-perspective analysis and statistical validation, some subjectivity in evaluation remains, introducing potential bias. A systematic methodology mitigates, but cannot eliminate, this risk. These limitations are common to all empirical AI research and do not undermine the validity of findings concerning evaluated model versions.



## 4 Requirements Extraction Analysis

This chapter presents Phase 1 findings, evaluating five large language models' capabilities in extracting and categorizing requirements from Software Requirements Specification (SRS) documents. This phase establishes fundamental competencies required for effective test case generation, identifying top-performing models for advancement to Phase 2 test case generation evaluation.

### 4.1 Experimental Setup

Phase 1 assessed extraction capabilities across seven experiments using diverse requirement documents. Models received complete SRS documents or specifications in multiple formats with instructions to extract both functional and non-functional requirements, ensuring proper categorisation and structured organisation. Experiments employed varied prompting strategies to evaluate robustness. Initial prompts were straightforward, while subsequent trials refined structure, corrected grammatical elements, and incorporated deep thinking techniques to encourage sophisticated analysis. This variation determined whether model performance stemmed from intrinsic capabilities or prompt engineering sensitivity.

Evaluation employed six metrics (100-point normalized scale): Coverage (percentage of requirements successfully identified), Clarity (readability and precision of formulations), Traceability (clear linkage to source material), Completeness (inclusion of all requirement types), Redundancy (absence of unnecessary duplication, inversely scored), and Hallucination (degree of fabricated content, inversely scored). Each metric was independently assessed through systematic comparison against manually derived ground truth reference sets, ensuring comprehensive and objective evaluation.

## 4.2 Overall Performance Summary

Analysis across all Phase 1 experiments reveals consistent performance patterns. Table 4.1 presents aggregated scores averaged across experiments, providing overall performance assessment for each model.

**Table 4.1:** Phase 1 Aggregate Performance Scores (0–100 scale)

| Model             | Coverage | Clarity | Traceability | Completeness | Redundancy | Hallucination | Average |
|-------------------|----------|---------|--------------|--------------|------------|---------------|---------|
| Claude AI         | 97.5     | 90.0    | 95.0         | 93.3         | 82.5       | 98.0          | 93.0    |
| Microsoft Copilot | 88.3     | 88.8    | 84.0         | 81.7         | 86.3       | 98.0          | 87.8    |
| DeepSeek          | 88.8     | 84.4    | 87.5         | 86.7         | 90.3       | 97.0          | 88.9    |
| ChatGPT           | 77.6     | 84.0    | 73.8         | 73.8         | 87.5       | 95.0          | 81.9    |
| Grok AI           | 83.8     | 78.1    | 76.3         | 77.5         | 80.5       | 95.0          | 81.9    |

Claude AI achieved the highest overall performance with a score of 93.0, excelling in Coverage (97.5), Traceability (95.0), and Completeness (93.3). Microsoft Copilot and DeepSeek performed similarly, with scores of 87.8 and 88.9, respectively. DeepSeek stood out for its strong performance in Redundancy avoidance (90.3), while Copilot displayed balanced performance across all metrics. On the other hand, ChatGPT and Grok AI scored substantially lower, both at 81.9. ChatGPT demonstrated particular weaknesses in Coverage (77.6

## 4.3 Key Findings by Metric

### 4.3.1 Coverage

Claude AI achieved the highest coverage (97.5), capturing nearly all functional and non-functional requirements, including edge cases and implicit specifications. It excelled at identifying requirements distributed across document sections and recognising non-functional requirements expressed indirectly through quality attribute discussions. DeepSeek (88.8) and Microsoft Copilot (88.3) performed well but occasionally missed requirements embedded in contextual discussions. DeepSeek tended toward over-consolidation, merging distinct requirements into single entries, whereas Copilot sometimes overlooked specialised categories, such as testing considerations and legal compliance. ChatGPT (77.6%) and Grok AI (83.8%) exhibited significant coverage limitations, frequently omitting business rules, assumptions, constraints, and specialised non-functional attributes.

### 4.3.2 Clarity

Microsoft Copilot achieved the highest clarity (88.8), producing well-structured outputs with precise categorisation and appropriate technical precision, striking a balance between conciseness and necessary detail. Claude AI (90.0) and ChatGPT (84.0) produced precise formulations, although Claude occasionally included unnecessary verbosity, while ChatGPT exhibited minor wording clarity issues. DeepSeek (84.4) generated readable outputs but sometimes suboptimally fragmented requirements. Grok AI scored the lowest (78.1), with outputs lacking consistent structure and including vague terminology such as "reasonable time frame" and "appropriate security measures," which compromises precision.

### 4.3.3 Traceability

Claude AI excelled (95.0), providing explicit section references, page numbers, and detailed hierarchical traceability matrices directly mapping requirements to source locations. DeepSeek (87.5) demonstrated strong traceability with clear attribution but less detailed mappings. Microsoft Copilot (84.0) provided adequate traceability using identifiers and general references, though with reduced precision. ChatGPT (73.8) and Grok AI (76.3) showed significant traceability weaknesses with vague or missing source references, making verification of extraction accuracy and completeness challenging.

### 4.3.4 Completeness

Claude AI achieved the highest completeness (93.3), systematically capturing all requirement categories and explicitly noting ambiguities requiring clarification. DeepSeek (86.7) provided reasonably complete extractions but occasionally missed specialised categories like audit logging and compliance requirements. Microsoft Copilot (81.7) sometimes overlooked business rules, assumptions, and constraints. ChatGPT (73.8) and Grok AI (77.5) showed notable completeness gaps, frequently omitting crucial non-functional categories, business rules, and contextual constraints.

### 4.3.5 Redundancy

DeepSeek achieved optimal redundancy control (90.3), producing concise extractions with minimal duplication. ChatGPT (87.5) and Microsoft Copilot (86.3) demonstrated reasonable control with occasional overlaps. Grok AI (80.5) exhibited moderate redundancy, particularly in business rules and role-specific descriptions. Claude AI scored lowest (82.5), occasionally repeating sub-requirements or presenting overlapping specifications. While ensuring thorough coverage, this increased documentation volume potentially complicates clarity and efficiency.

### 4.3.6 Hallucination

All models demonstrated low hallucination rates. Claude AI (98.0), Microsoft Copilot (98.0), and DeepSeek (97.0) exhibited strong discipline in grounding extractions in actual content, typically flagging uncertainties explicitly. ChatGPT (95.0) and Grok AI (95.0) showed slightly elevated tendencies, occasionally generating requirements inferred from domain knowledge rather than document content, risking discrepancies between documented intentions and extractions.

## 4.4 Critical Observations

Several cross-cutting patterns emerged from Phase 1 analysis:

**Error-Handling Gaps:** All models struggled with error-handling use cases, often missing requirements related to failure scenarios, exception processing, and recovery procedures. This consistent gap suggests error conditions receive insufficient emphasis in standard requirement formulations, indicating the need for explicit attention during specification and extraction activities.

**Compliance Challenges:** Compliance requirements posed difficulties across models. While Microsoft Copilot occasionally identified requirements like GDPR anonymisation or PCI-DSS standards, other models frequently overlooked legal, regulatory, and compliance specifications, particularly in domains where compliance is critical.

**Compliance Challenges:** Compliance requirements posed difficulties across models. While Microsoft Copilot occasionally identified

requirements like GDPR anonymisation or PCI-DSS standards, other models frequently overlooked legal, regulatory, and compliance specifications, particularly in domains where compliance is critical.

**Prompting Impact:** Prompting strategy impacted performance, though intrinsic capability differences proved more significant. All models performed better with structured, detailed prompts versus casual instructions. However, poor-performing models remained ineffective despite optimised prompts, while strong models maintained high performance even with suboptimal instructions.

## 4.5 Visual Processing Limitations

A significant limitation emerged during experiments involving non-textual requirement sources (diagrams, use case visualizations, class diagrams). All models showed substantially reduced performance processing visual information compared to text-based sources.

Diagram Processing Challenges included: Text Extraction Without Context. Models extracted text labels and annotations but struggled to understand structural relationships (arrows, hierarchies, spatial arrangements) crucial for interpretation. In use case diagrams, models identified actor names and use case labels but had difficulty comprehending "extends" and "includes" relationships. Relationship Misinterpretation class diagrams presented particular challenges. Models identified class names and occasionally method signatures, but often failed to interpret multiplicities, inheritance, and associations. One-to-many relationships between entities were frequently overlooked or misinterpreted. Complete Visual Omission models sometimes completely ignored diagrams when accompanying text was present, suggesting bias toward text-based data, potentially due to training optimisation.

This visual processing gap constitutes a significant limitation for workflows relying on diagrammatic modelling languages (UML, SysML, BPMN) for requirements engineering. Organisations that use visual specifications require either manual diagram interpretation or specialised diagram-to-text conversion preprocessing to supplement AI-based requirements extraction.

## 4.6 Model Selection for Phase 2

Based on Phase 1 performance, three models advanced to Phase 2: Claude AI, Microsoft Copilot, and ChatGPT. Selection criteria focused on overall performance, requirements comprehension, and practical utility for testing workflows.

Claude AI emerged as top performer with highest overall scores and exceptional strengths in coverage, traceability, and completeness. Strong requirement extraction capabilities indicated solid foundation for generating test cases requiring deep requirements understanding.

Microsoft Copilot was selected for balanced performance across all metrics with particular clarity strength. Its ability producing well-structured, actionable outputs made it promising candidate for generating usable test specifications.

ChatGPT, despite lower Phase 1 scores, was included due to widespread adoption and baseline competency. Phase 2 would determine whether its requirement extraction limitations hindered test generation capabilities or if it could compensate through other strengths.

DeepSeek and Grok AI did not progress to Phase 2. Despite DeepSeek's competitive overall score, its over-consolidation tendency and occasional completeness gaps raised concerns about comprehensive test case generation ability. Grok AI's clarity and traceability weaknesses suggested fundamental limitations likely affecting test specification quality. Resource constraints necessitated focusing on most promising candidates.

ShareProject content  
 Research Project Master-Created by you  
 Report of comparision - Expirement 9 - Stage 1 - Testing Part.docx  
 131 lines  
 docx  
 Report of comparision - Expirement 8 - Stage 1 - Testing Part.docx  
 91 lines  
 docx  
 Final Analysis - Expirement 7 - Stage 1 - Testing Part.docx  
 95 lines  
 docx

## 5 Test Case Generation Evaluation

Building upon requirements extraction capabilities established in Phase 1, Phase 2 evaluates models' ability to generate high-quality test cases through three-stage progressive evaluation framework.

### 5.1 Exploratory Five-Model Assessment

Stage 1 conducted exploratory experiments evaluating test case generation capabilities of all five models: ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude. Experiments utilised varied prompting strategies and testing scenarios to establish baseline performance, identifying strengths and weaknesses in test case generation.

#### 5.1.1 Experimental Approach

Experiments varied prompt complexity and specificity to evaluate model robustness. Initial experiments employed simple prompts requesting basic happy-path test cases. Progressive sophistication introduced positive and negative scenarios, boundary value testing, and comprehensive functional/non-functional requirement coverage. Advanced experiments focused on requirements traceability matrices, formal test design techniques, and IEEE 829 compliance. This variation determined whether models required extensive prompt engineering for acceptable performance or demonstrated intrinsic test generation capabilities resilient to prompt quality variations.

#### 5.1.2 Aggregated Performance Analysis

Table 5.1 presents Phase 1 aggregate performance across experiments and prompting strategies.

Microsoft Copilot and Claude emerged as the top performers, with nearly identical overall scores (91.4 and 91.2, respectively). Both demonstrated professional-quality test case generation, each with distinct complementary strengths. ChatGPT achieved respectable performance (88.3), but showed notable gaps in non-functional requirement coverage and integration testing. Grok and DeepSeek scored

**Table 5.1:** Phase 1 Average Performance Scores

| Model             | Overall Score | Key Strengths                                       | Primary Weaknesses               |
|-------------------|---------------|-----------------------------------------------------|----------------------------------|
| Microsoft Copilot | 91.4          | Technical precision, actionable steps, NFR coverage | Occasional breadth limitations   |
| Claude AI         | 91.2          | Comprehensive coverage, formal structure, RTM       | Slight verbosity                 |
| ChatGPT           | 88.3          | Clear presentation, traceability                    | Limited NFR depth                |
| Grok              | 77.4          | Modular organization                                | Weak coverage, minimal negatives |
| DeepSeek          | –             | Weakest – Consistently shallow outputs              | –                                |

substantially lower, revealing fundamental limitations in test comprehensiveness and depth, which hindered their ability to generate high-quality, thorough test cases.

### 5.1.3 Key Model Characteristics

**Microsoft Copilot** demonstrated exceptional balance between functional and non-functional requirement coverage, excelling in technically precise test specifications with clear validation criteria. Test cases included specific elements like HTTP status codes, API response formats, database state expectations, and quantified performance thresholds. Copilot effectively utilized formal test design techniques (boundary value analysis, equivalence partitioning, decision tables, state transition testing) and gave particular attention to security testing, generating explicit scenarios for authentication bypass, SQL injection, cross-site scripting, and data exposure vulnerabilities.

**Claude AI** produced broadest test coverage with systematic requirement traceability, generating comprehensive test suites spanning all functional areas while clearly mapping test cases to SRS requirements using formal traceability matrices. Test case structure adhered closely to IEEE 829 standards, including complete preconditions, detailed test steps, specific test data, measurable expected results, and



clear pass/fail criteria. Claude showed particular strength identifying edge cases and covering negative scenarios, systematically exploring potential failure modes and error conditions.

**ChatGPT** generated well-structured test cases with clear presentation and logical organisation, providing good functional coverage with explicit requirement mappings. However, non-functional requirement coverage was less comprehensive than leading models. Security scenarios, performance validation, and integration testing received insufficient attention, with test data specifications sometimes generic rather than concrete.

**Grok and DeepSeek** exhibited fundamental limitations preventing advancement. Grok demonstrated basic functional test generation but lacked depth, focusing on happy-path scenarios with minimal negative testing and weak requirements traceability. DeepSeek consistently produced weakest outputs with narrow functional coverage, minimal non-functional requirement attention, and insufficient execution detail, requiring substantial human supplementation to reach acceptable quality.

### 5.1.4 Conclusions and Model Selection

Stage 1 established a clear performance hierarchy. Microsoft Copilot and Claude demonstrated professional-level test generation capabilities, qualifying for a detailed Stage 2 evaluation. ChatGPT achieved acceptable baseline performance, justifying Stage 2 inclusion to determine whether requirement extraction limitations constrained test generation or if other strengths could compensate. Grok and DeepSeek exhibited fundamental output limitations, preventing advancement to systematic evaluation phases.

## 5.2 Systematic Three-Model Comparison

Stage 2 conducted ten controlled experiments systematically evaluating three selected models: Microsoft Copilot, Claude, and ChatGPT. Experiments employed consistent prompting methodology and evaluation criteria across various CORADS modules and testing scenarios.

### 5.2.1 Experimental Framework

All Stage 2 experiments used identical structured prompts with a persona-based approach: "Act as a Principal Quality Engineer with 20+ years of experience." Prompts specified an IEEE 829-compliant test case structure, including test ID, description, preconditions, test steps, test data, expected results, postconditions, pass/fail criteria, priority, and test environment. Special emphasis was ensured to provide comprehensive coverage, including edge cases, negative scenarios, security testing, and performance validation.

Evaluation employed seven criteria (100-point scales): Accuracy (alignment with SRS requirements), Completeness (inclusion of required test case components), Test Coverage (breadth across functional/non-functional requirements), Efficiency (appropriate test case count avoiding redundancy), Quality (logical clarity and execution detail), Bug Detection (effectiveness revealing potential defects), and Usability (clarity for manual and automated testing).

### 5.2.2 Initial Three-Model Comparison

The first four experiments evaluated all three models across core CORADS modules to establish relative performance and identify candidates for elimination.

**Table 5.2:** Phase 2 Experiments 1–4 Results

| Experiment     | Module              | Claude AI   | Microsoft Copilot | ChatGPT     | Winner |
|----------------|---------------------|-------------|-------------------|-------------|--------|
| Exp 1          | Driver Registration | 94.3        | 89.3              | 75.7        | Claude |
| Exp 2          | Campaign Management | 100.0       | 98.6              | 64.3        | Claude |
| Exp 3          | Payment System      | 98.6        | 92.9              | 70.0        | Claude |
| Exp 4          | Real-Time Tracking  | 96.4        | 92.9              | 82.1        | Claude |
| <b>Average</b> | –                   | <b>97.3</b> | <b>93.4</b>       | <b>73.0</b> | Claude |

Claude achieved the highest scores across all four experiments (average 97.3), demonstrating exceptional test case generation performance. Microsoft Copilot performed strongly with a consistent average of 93.4. ChatGPT scored substantially lower (73.0), exhibiting

significant limitations in test coverage (particularly non-functional requirements), completeness (missing essential test case components), and bug detection capability (focusing on happy paths, lacking edge case and failure scenario coverage).

**ChatGPT Elimination:** After Experiment 4, ChatGPT was formally eliminated from further Stage 2 evaluation. Consistent under-performance across critical dimensions suggested fundamental limitations unaddressable through prompt engineering alone. Key issues included coverage deficiencies (20-30 points below leading models), non-functional requirement weaknesses (minimal security and performance testing), structural limitations (incomplete test case components), and insufficient depth (generic specifications lacking concrete details).

### 5.3 Focused Two-Model Evaluation

Experiments 5 through 10 compared Claude and Microsoft Copilot exclusively, exploring specialized testing scenarios and additional CORADS functionality.

**Table 5.3:** Phase 2 Experiments 5–10 Results

| Exp            | Module                           | Claude AI   | Microsoft Copilot | Winner  |
|----------------|----------------------------------|-------------|-------------------|---------|
| Exp 5          | Driver Registration (Security)   | 80.0        | 86.7              | Copilot |
| Exp 6          | Campaign Management (Advanced)   | 86.7        | 93.3              | Copilot |
| Exp 7          | Payment System (Integration)     | 90.0        | 77.0              | Claude  |
| Exp 8          | Real-Time Tracking (Performance) | 84.0        | 90.0              | Copilot |
| Exp 9          | Security & Edge Cases            | 90.0        | 83.0              | Claude  |
| Exp 10         | Comprehensive (All Modules)      | 92.0        | 85.7              | Claude  |
| <b>Average</b> | –                                | <b>87.1</b> | <b>86.0</b>       | –       |

Experiments 5-10 revealed highly competitive performance, with Claude averaging 87.1 and Copilot 86.0. Each model showcased strengths in different scenario types, highlighting complementary capabilities.

Microsoft Copilot excelled in security-focused testing (Exp 5), complex business logic handling (Exp 6), and performance validation (Exp 8) with technical precision and quantifiable metrics. Claude demonstrated superiority in integration testing (Exp 7), comprehensive edge case coverage (Exp 9), and full-system evaluation (Exp 10) through systematic thoroughness and strong requirements traceability.

### 5.4 Comparative Analysis

**Claude’s Approach:** Methodical requirements analysis exhaustively identifying all testable conditions, generating corresponding test cases with clear traceability. This thoroughness ensures comprehensive coverage encompassing security, performance, reliability, usability, and functional validation. While producing large test suites requiring more execution effort, Claude prioritizes completeness, ensuring no system aspect is overlooked. Systematic thoroughness makes it particularly well-suited for comprehensive testing scenarios, though demanding more time and processing resources.

**Microsoft Copilot’s Approach:** Focuses on generating technically precise, immediately actionable test specifications. Test cases include specific technical details (API endpoints, HTTP status codes, JWT token handling, database transaction expectations, protocol specifications) highly valuable for developers and automation engineers. This precision enables easier test automation and simplifies technical troubleshooting. Copilot balances coverage and efficiency, generating more concise test suites through intelligent scenario consolidation, making it ideal for environments prioritizing technical accuracy and automation.

#### 5.4.1 Overall Assessment and Claude Selection

Aggregating across all ten Phase 2 experiments yields overall performance assessment.

**Table 5.4:** Phase 2 Overall Performance

| Model             | Experiments<br>1–4 Avg | Experiments<br>5–10 Avg | Overall Avg | Total Wins |
|-------------------|------------------------|-------------------------|-------------|------------|
| Claude AI         | 97.3                   | 87.1                    | 91.2        | 7 of 10    |
| Microsoft Copilot | 93.4                   | 86.0                    | 88.9        | 3 of 10    |
| ChatGPT           | 73.0                   | –                       | 73.0        | 0 of 4     |

Claude achieved highest overall Stage 2 performance (91.2), showcasing exceptional results in early experiments and maintaining competitive performance in later stages, winning 7 of 10 experiments. Microsoft Copilot demonstrated strong performance (88.9), offering particular advantages for automation-focused workflows and technically oriented testing teams due to precision and efficiency.

Claude’s Selection for Stage 3 was driven by: Highest Overall Performance indicating most consistent capability across various testing scenarios; Superior Comprehensive Coverage essential for thorough system validation; Professional QA Alignment systematic approach aligned with professional quality assurance practices, emphasizing auditability and detailed requirements traceability; Balanced Strengths balanced performance across all evaluation criteria suggesting fewer systematic blind spots, making it most well-rounded model for evaluation against human-generated test cases.

## 6 Comprehensive Human vs. AI Evaluation

This chapter presents Stage 3 findings comparing Claude, the highest-performing AI model from Stage 2, against professional human quality assurance engineers through fifteen controlled experiments. The investigation gathered detailed empirical evidence regarding where AI excels, where it falls short, and how AI and human capabilities might complement each other in practical testing workflows. Experiments progressed from exploratory assessment (Experiments 1-8) to controlled refinements (Experiments 9-15), systematically addressing identified limitations while building comprehensive understanding of strengths and weaknesses for both AI and human testers.

### 6.1 Experimental Design

Stage 3 employed a systematic methodology comparing Claude-generated test cases with those written by human QA professionals across multiple dimensions. Human test cases were developed by experienced professionals who created documentation for CORADS using the same SRS provided to Claude, ensuring fair comparison based on identical requirement specifications.

#### 6.1.1 Evaluation Framework

Stage 3 extended evaluation beyond Stage 2 criteria by incorporating expanded metrics addressing specific production testing environment concerns: Accuracy (factual correctness, percentage of test specifications aligning with SRS requirements without hallucinated content), Completeness (breadth of coverage, percentage of requirements receiving appropriate test coverage including functional and non-functional specifications), Requirements Traceability Matrix (RTM) Coverage (percentage of requirements explicitly linked to test cases through requirement identifiers), Test Coverage (systematic attention across requirement categories, testing techniques, and scenario types), Oracle Specificity (1-5 scale measuring precision of expected results, evaluating measurability and objectivity versus vagueness), Hallucination Rate (fabricated specifications per 100 test steps unsupported

by SRS documentation), Duplicate Reduction (percentage of redundant test cases identified and eliminated), Risk Prioritization Alignment (Cohen's kappa measuring agreement between AI-generated and human-generated priority assignments), and Efficiency (time and effort required to generate test cases).

### **6.1.2 Dual-Perspective Verification and Statistical Methods**

All Stage 3 experiments employed dual-perspective analysis combining the researcher's independent evaluation with LLM-assisted verification. Two independent human raters evaluated all test cases using structured rubrics, ensuring consistency and objectivity. Cohen's kappa coefficient measured inter-rater reliability, with a target threshold of 0.70, indicating substantial agreement. Statistical analysis included paired t-tests and Wilcoxon signed-rank tests assessing significance at a  $p < 0.05$  threshold. Cohen's d quantifies the effect size, providing insight into the magnitude of performance differences.

### **6.1.3 Experimental Progression**

Experiments 1-8 took exploratory approach identifying recurring patterns in Claude's strengths and limitations. Early findings highlighted hallucinations (invented scenarios unsupported by SRS content), traceability gaps despite overall strong performance, and risk prioritization misalignment with human business intuition.

Experiments 9-15 implemented controlled refinements addressing these issues specifically: explicit grounding constraints instructing Claude to mark assumptions extending beyond SRS as 'out of scope', RTM-first templates requiring explicit requirement mapping before generating test steps, screen/element validation ensuring UI references matched specifications, duplicate elimination passes avoiding redundancy, policy-citation rules demanding explicit source references, oracle contract schemas enforcing measurable expectations, and risk weighting models aligning prioritization with business criticality.

## 6.2 Overall Quantitative Results

Table 6.1 presents aggregate performance across all fifteen Phase 3 experiments.

**Table 6.1:** Stage 3 Overall Performance Summary

| Metric                       | Claude AI Avg | Human Avg | $\Delta$ (Claude – Human) | Interpretation             |
|------------------------------|---------------|-----------|---------------------------|----------------------------|
| Accuracy %                   | 96.2          | 92.8      | +3.4                      | AI fewer factual errors    |
| Completeness %               | 72.8          | 87.1      | -14.3                     | Humans infer missing FRs   |
| RTM Coverage %               | 100.0         | 55.0      | +45.0                     | AI perfectly traceable     |
| Test Coverage %              | 95.0          | 88.0      | +7.0                      | AI more systematic         |
| Oracle Specificity (1–5)     | 5.0           | 1.8       | +3.2                      | AI measurable expectations |
| Hallucination Rate (per 100) | 0.55          | 0.88      | -0.33                     | Lower AI over-inference    |
| Duplicate Reduction %        | 50.0          | 0.0       | +50.0                     | AI identifies redundancy   |
| Efficiency (relative)        | 8× faster     | 1×        | –                         | Major productivity gain    |

Claude demonstrated superiority in accuracy, RTM coverage, test coverage, oracle specificity, and efficiency. However, human testers outperformed Claude in completeness, leveraging contextual inference to identify implicit requirements Claude missed. Statistical significance testing confirmed key differences: Claude significant advantage in accuracy ( $p = 0.012$ , Cohen’s  $d = 1.01$ , large positive effect), RTM coverage ( $p < 0.001$ , Cohen’s  $d = 2.3$ ), and efficiency ( $p < 0.001$ ). Interrater reliability achieved  $\kappa = 0.72$ , surpassing target 0.70 threshold, indicating substantial agreement.

## 6.3 Key Findings by Dimension

### 6.3.1 Accuracy and Hallucination Control

Claude achieved 96.2% accuracy, outperforming human testers at 92.8%, producing fewer factual errors and SRS misalignments. Early experiments revealed hallucination concerns where Claude generated scenarios, data fields, or workflows unsupported by documentation. Initial hallucination rate was 0.92 fabricated specifications per 100 test steps. Grounding constraints implemented in Experiments 9-15



reduced this to 0.55 per 100 test steps (40% reduction). Remaining hallucinations primarily involved reasonable inferences about error handling and edge cases where SRS was silent. Human testers exhibited 0.88 hallucination rate, slightly higher than refined Claude, though human "over-inference" manifested differently through experience-based assumptions about operational procedures, organizational policies, or technical implementations.

### 6.3.2 Completeness and Contextual Understanding

Human testers achieved 87.1% completeness, outperforming Claude's 72.8%, demonstrating a superior ability to infer implicit requirements and contextually implied functionality. Professional testers recognized standard functionality commonly omitted from SRS documentation (comprehensive logout testing, session timeout validation, concurrent login handling). Business logic interpretation favoured humans, as SRS described payment processing without specifying failure handling, recovery procedures, or reconciliation workflows. Consequently, human testers proactively generated comprehensive failure scenarios based on standard financial transaction practices. Domain knowledge enabled humans to identify missing but critical test coverage for regulatory compliance, accessibility requirements, and production operational concerns. Claude demonstrated systematic coverage of stated requirements but lacked initiative identifying specification omissions.

### 6.3.3 Requirements Traceability

Claude achieved a perfect 100% RTM coverage, with explicit requirement identifiers for every test case, ensuring that each test was clearly linked to a corresponding requirement. In contrast, human test suites reached only 55% RTM coverage, with 25.4% of test cases lacking any explicit requirement traceability.

Experiment 10 specifically evaluated traceability through the Claude achieved perfect 100% RTM coverage with explicit requirement identifiers for every test case. Human test suites reached only 55% RTM coverage, with 25.4% of test cases lacking explicit requirement traceability.

Claude produced comprehensive mapping generating 96 test cases covering all 87 functional requirements with explicit  $FR_I Dtags$  (averaging 1.1 tests per requirement).

### 6.3.4 Oracle Quality and Measurability

Oracle specificity revealed dramatic differences. Claude consistently achieved perfect 5.0 rating, generating measurable and objective expected results. Human test cases averaged 1.8/5.0, with 19% exhibiting vague, unverifiable expectations. Claude-generated test cases specified clear, measurable expectations: observable behaviors (UI state changes, system responses, database modifications), precise thresholds (response time < 2.0 seconds, success rate 99.5%), specific system codes (HTTP 201 Created, HTTP 401 Unauthorized), and data side-effects (table.field = expected\_value\_with\_concrete\_values). Human test cases often used vague expectations: "system works correctly," "appropriate error message displayed," or "data saved successfully." These generated tests enable objective and repeatable validation, while human-generated tests sometimes

### 6.3.5 Risk Prioritization and Business Judgment

Risk prioritisation emerged as Claude's most significant limitation, with a Cohen's kappa of 0.34 indicating only fair agreement, revealing a systematic misalignment. Claude employed a logical but context-insensitive approach: High priority to functional correctness and data integrity concerns, Medium priority to performance requirements unless explicit SLA specifications were given, and Low to Medium priority to UI/UX issues. Human testers applied their experienced judgment, considering revenue impact (payment processing was given the highest priority, regardless of technical complexity), regulatory requirements (security and compliance were marked as critical), user base scale (features affecting all users were prioritised over administrative functions), and historical defect patterns (areas with past production issues received heightened attention). This limitation stems from Claude's lack of organizational context, business strategy understanding, and historical operational knowledge, critical elements senior QA professionals bring through deep domain expertise.

### 6.3.6 Test Suite Structure and Redundancy

Claude demonstrated superior duplicate identification and elimination. Initially 50% of generated test cases contained redundancy (testing functionally identical conditions with superficial variations). Upon review, Claude successfully identified and consolidated redundant

test cases. Human test suites exhibited minimal redundancy (0% measured), partially due to conservative approach prioritizing fewer, more focused test cases. While avoiding redundancy, this sometimes resulted in undercoverage of specific requirements. Human testers achieved efficiency by reducing test case numbers, occasionally at cost of full coverage, rather than intelligent consolidation of redundant tests.

### 6.3.7 Efficiency and Productivity

Efficiency differences were dramatic. Claude generated comprehensive test suites in approximately 6 minutes per module versus human average of 65 minutes per module, representing 10× productivity difference. For complete CORADS system, Claude generated entire test suite in under one hour versus multiple days of professional QA engineer effort. However, comparing raw generation speed oversimplifies value assessment. Human time investment involved contextual analysis, business logic reasoning, and domain expertise application, cognitive tasks not directly measurable in time spent, partially orthogonal to mechanical test case generation.

## 6.4 Qualitative Observations and Patterns

### Complementary Strengths and Weaknesses

**Claude's Strengths:** Systematic comprehensiveness ensuring no stated requirement unaddressed, structural consistency providing uniform quality, documentation completeness adhering to IEEE 829 standard, technical precision specifying exact validation criteria, exemplary traceability discipline maintaining perfect requirement mapping, and objectivity free from bias or time pressures.

**Claude's Weaknesses:** Contextual blindness preventing inference of implicit requirements, business logic naivety hindering domain-specific workflow understanding, risk insensitivity lacking business context for priority assignments, environmental unawareness omitting platform-specific and operational concerns, and over-literal interpretation missing unspoken expectations.

**Human Strengths:** Contextual reasoning filling SRS gaps with reasonable assumptions, business judgment prioritizing based on potential impact understanding, domain expertise identifying standard functionality omitted from documentation, risk awareness focusing on critical areas, execution pragmatism balancing thoroughness with available resources, and environmental sensitivity addressing platform-specific and operational concerns.

**Human Weaknesses:** Inconsistency causing quality variation across testers and time periods, documentation gaps leaving incomplete traceability, bias influencing coverage toward familiar or preferred areas, efficiency limitations constraining thoroughness, and oracle vagueness reducing repeatability and automation readiness.

#### 6.4.1 Recommended Hybrid Workflow

Analysis suggests neither pure AI nor pure human approach achieves optimal results. Evidence supports hybrid workflow combining complementary strengths:

**Phase A: AI-Driven Draft Generation** - Claude generates comprehensive baseline test suite with perfect traceability, measurable oracles, and systematic coverage of stated requirements. Deliverables include complete IEEE 829 test cases, requirements traceability matrix (RTM), and coverage dashboard identifying tested and untested requirements.

**Phase B: Human Expert Validation and Enhancement** - Professional QA engineers review AI-generated suite, applying contextual knowledge to identify missing implicit requirements, adjust risk priorities based on business impact, add environment-specific scenarios reflecting deployment realities, refine test data selections based on production patterns, and validate business logic interpretation accuracy.

**Phase C: AI-Assisted Analysis and Maintenance** - Claude performs consistency checking across enhanced suite, identifies remaining gaps through RTM analysis, suggests additional scenarios for incomplete coverage areas, maintains traceability as requirements evolve, and generates regression test selection recommendations.

This hybrid approach achieved 96% accuracy, 95% coverage, < 1% hallucination rate, and 8× efficiency improvement compared to purely human workflows while preserving human judgment advantages.

## Statistical Validation

Comprehensive statistical analysis validated Phase 3 findings:

**Primary Outcome (Accuracy):** 95% confidence interval for Claude accuracy =  $96.2 \pm 2.8\%$ . Significance testing versus human baseline yielded  $p = 0.012$  ( $p < 0.05$ ), confirming statistically significant advantage. Effect size Cohen's  $d = 1.01$  represents large positive effect, indicating not merely statistically significant but practically meaningful improvement.

**Inter-Rater Reliability:** Cohen's  $\kappa = 0.72$  exceeds target 0.70 threshold, demonstrating substantial inter-rater agreement and validating evaluation methodology reliability.

**Secondary Outcomes:** RTM coverage advantage achieved  $p < 0.001$  with Cohen's  $d = 2.3$  (very large effect). Oracle specificity difference reached  $p < 0.001$  with Cohen's  $d = 1.8$  (large effect). Completeness disadvantage measured  $p = 0.03$  with Cohen's  $d = 0.6$  (medium effect).

Statistical rigor establishes Phase 3 findings meet academic and industrial credibility thresholds for reproducible scientific conclusions.

## 7 Challenges and Limitations

This chapter explores challenges encountered during research and inherent limitations of both AI-based test generation and the research methodology itself. Understanding these constraints is crucial for accurately interpreting findings, recognizing generalizability limitations, and identifying potential directions for future investigation. The analysis distinguishes between AI-specific technical limitations, methodological research constraints, and practical implementation challenges organizations may face when deploying such systems.

### 7.1 AI-Specific Technical Challenges

Large language model test generation exhibits characteristic limitations stemming from fundamental AI capabilities and training approaches. These technical challenges manifested consistently across experiments despite prompt engineering efforts and controlled refinements.

#### 7.1.1 Hallucination and Over-Inference

Hallucination, or generation of content unsupported by source documentation, represents one of the most critical limitations. In early Phase 3 experiments, Claude demonstrated initial hallucination rate of 0.92 fabricated specifications per 100 test steps. While controlled refinements (grounding constraints) reduced this to 0.55, complete elimination proved unattainable.

Hallucination severity varied. Minor cases involved reasonable assumptions about standard functionality (logout confirmation dialogues, session timeout handling not explicitly mentioned in SRS). While technically hallucinations, these inferences often proved valuable for test completeness. More problematic cases involved entirely fictional requirements. Claude generated "urban/rural driver classification" with associated differential pricing and route optimisation logic, which was completely absent from CORADS SRS, posing scope creep risk, development misalignment, and wasted resource risk.

Documentation quality affects hallucination frequency: well-specified requirements reduce hallucinations because models find explicit guidance, but ambiguous or incomplete specifications trigger inference mechanisms, increasing hallucination likelihood. This creates a paradox, AI assistance seems most valuable with poor documentation, yet performs most reliably when specifications are comprehensive.

Experiment 13's resilience testing highlighted systematic over-inference. Claude generated 28 fault-tolerance test scenarios, of which only 2 (7%) were explicitly SRS-supported. The remaining 24 represented industry best practices (retry mechanisms, exponential backoff, failover strategies, caching implementations) inferred from domain knowledge despite SRS silence. While potentially identifying valuable system improvements, these hypothetical requirements could confuse development priorities and misdirect testing efforts.

### 7.1.2 Contextual Understanding Limitations

AI models demonstrate superficial rather than deep contextual understanding, manifesting in several quality-affecting ways:

**Implicit Requirement Recognition:** Human testers excelled identifying unstated but necessary functionality based on domain expertise (comprehensive logout workflows, concurrent login handling, session management) despite minimal SRS coverage. Claude generated these inconsistently, requiring explicit specification.

**Business Logic Interpretation:** When SRS described payment processing without detailed failure handling specifications, human testers drew on financial transaction domain knowledge to generate comprehensive failure scenarios. Claude addressed explicitly stated error conditions but lacked initiative exploring unstated yet practically necessary failure modes, underscoring human ability recognizing business-critical scenarios when documentation is inadequate.

**End-to-End Flow Understanding:** Claude demonstrated difficulties with sequencing dependencies in complex workflows, occasionally mis-sequencing test steps in multi-stage processes (attempting operations before prerequisite conditions were established). Human testers naturally recognized logical flow requirements and temporal dependencies even when specifications presented requirements non-sequentially.

**Integration Testing Gaps:** Claude omitted integration testing entirely despite CORADS comprising multiple interconnected modules. Human test suite included 33 integration tests covering Firebase communication, server interactions, network handling, local database synchronization, and push notification delivery. Claude's focus on individual requirement validation prevented recognizing critical inter-module validation needs.

### 7.1.3 Risk Prioritization Deficiencies

Risk-based testing prioritization proved Claude's most significant limitation with only fair agreement ( $\kappa = 0.34$ ) between its priority assignments and human professional judgment. Claude relied on systematic but context-insensitive algorithm: functional correctness and data integrity marked High, performance requirements Medium (unless explicit SLAs provided), and UI/UX issues Low to Medium. While logical and consistent, this lacked business context understanding.

Human testers applied experienced judgment considering revenue impact (payment processing prioritized high regardless of technical complexity), regulatory requirements (security and compliance marked critical due to regulatory exposure), user base scale (features affecting all users prioritized over administrative functions), and historical defect patterns (areas with past production issues receiving heightened attention).

Misalignments included profile customization (Claude Medium, humans Low), OTP verification (Claude uniformly High, humans differentiated based on authentication criticality), and reporting features (Claude uniform Medium, humans prioritized revenue-impact reports High and informational dashboards Low). These discrepancies stem from Claude's lack of organizational context, business strategy understanding, and historical operational data, critical insights senior QA professionals bring through domain expertise.

### 7.1.4 Environmental and Operational Unawareness

AI-generated tests had limited understanding of real-world deployment environments and operational challenges. Human testers considered ensuring app works across different Android versions (7-10),



testing behavior on various browsers (Chrome, Opera, Edge, Firefox), and evaluating performance under different mobile network conditions and GPS accuracy issues.

Claude generated technically accurate functional tests but often missed environmental variation testing unless specifically instructed. It didn't focus on production-related concerns (deployment verification, monitoring tool integration, log analysis support, rollback scenario handling). Human testers naturally incorporated these aspects through real-world experience deploying and supporting systems in live environments.

## **7.2 Methodological Limitations**

The research methodology, while systematic and rigorous, exhibits inherent limitations affecting findings generalizability and interpretation.

### **7.2.1 Single SRS Document Constraint**

The investigation utilised a single SRS document (CORADS) as the primary experimental artefact. While practical for enabling controlled comparisons, this limits the generalizability of findings. CORADS represents a specific domain (location-based advertising), medium-scale web and mobile platform, and a professional IEEE-compliant specification with certain documentation quality and requirement characteristics.

In different domains, AI performance might vary significantly. Safety-critical systems (medical devices, aviation, automotive) require more exhaustive validation with varying philosophies of testing than commercial web applications. Highly regulated domains (financial services, healthcare, government) require specific compliance testing and audit trails, potentially highlighting different AI strengths. Embedded systems, real-time systems, and algorithm-intensive applications present unique testing challenges that potentially influence AI performance.

Specification quality plays a crucial role. This research employed professionally written, well-structured IEEE-standard-adherent SRS

enabling AI optimal performance. Organisations with informal requirements documentation (user-story-based specifications, poorly maintained requirements) may see different effectiveness levels. Conversely, organisations with more rigorous specification practices might see improved AI performance, as more precise and detailed specifications enable better AI test case generation.

### **7.2.2 Model Version Temporality**

Research evaluated specific AI model versions available during investigation period (late 2024 to early 2025). Given large language models evolve rapidly through continuous training, architectural improvements, and specialized fine-tuning, findings represent performance snapshots potentially differing from earlier versions or future releases.

This temporality influences both absolute performance assessments and relative model rankings observed. While Claude demonstrated superiority during evaluation, updated versions of Microsoft Copilot, ChatGPT, or others could produce different results. Additionally, entirely new model families might emerge exhibiting different capability profiles. Findings should be viewed as current state-of-the-art reflection rather than definitive, permanent AI technology limitations.

### **7.2.3 Evaluation Subjectivity**

Despite dual-perspective analysis, structured rubrics, and statistical validation, the evaluation involved some subjective judgment. Assessing test case quality requires interpretation, particularly when considering appropriateness, practical utility, and professional standards alignment. Different evaluators prioritise various quality dimensions or apply different acceptability thresholds.

Inter-rater reliability ( $= 0.72$ ) indicates substantial agreement but also suggests perfect consistency was not achieved. The 28% unexplained variance highlights legitimate quality assessment subjectivity. While evaluation rubrics provided guidance, they couldn't eliminate judgment calls in borderline situations, trade-off decisions, or determining specific criteria relative importance. This inherent subjectivity reflects test case evaluation complexity, where different perspectives and priorities influence the final assessment.

#### **7.2.4 Training Data Quality and Adversarial Risks**

The research identified two interconnected concerns regarding the foundation of generative AI model capabilities: training data quality and susceptibility to data manipulation.

##### **Public Data Quality Constraints**

LLMs trained primarily on publicly available text exhibit significant quality limitations in technical and domain-specific contexts, particularly evident in software requirements engineering and test case generation where training data gaps versus professional standards become apparent. Publicly available SRS documents come predominantly from academic course projects/student assignments, open-source project documentation (often informal or incomplete), small organizations with limited QA resources, and tutorials/educational materials prioritizing teaching over professional rigor.

High-quality SRS documents from large enterprises (Fortune 500 companies, government contractors, safety-critical system developers) remain proprietary and training-inaccessible. These typically include detailed database schemas and data dictionaries, comprehensive API contracts with request/response examples, system architecture diagrams with deployment details, non-functional requirements with measurable SLAs, and regulatory compliance mappings with audit trail specifications. Since these high-quality documents aren't part of training data, models cannot replicate technical detail level, structural rigor, and domain-specific terminology characterizing professional software engineering, directly limiting professional-grade test artifact generation ability.

##### **Manipulation and Adversarial Risks**

Reliance on publicly available data introduces intentional manipulation risk. Since models learn from any accessible text, malicious actors could inject misleading or harmful information into public sources, influencing model behaviour. Examples include outdated best practices (models occasionally recommending deprecated testing approaches or obsolete security practices likely learned from older documentation

remaining publicly accessible despite being outdated) and adversarial content injection risk (though not directly observed, a theoretical vulnerability exists where bad actors could systematically publish intentionally flawed SRS examples or test case templates designed to degrade model performance).

### **7.2.5 Human Baseline Variability**

Human-generated test cases used as a comparison baseline exhibited inherent variability. Test cases came from experienced QA professionals, but individual differences in experience levels, domain expertise, organisational context, and testing philosophy influenced outputs. One tester might emphasise comprehensive edge case coverage while another prioritises pragmatic execution efficiency. This variability introduces uncertainty when determining whether observed differences reflect AI limitations or human tester-specific characteristics. While inter-rater reliability measures partially addressed this concern, complete elimination of human baseline variability remains impossible, representing an inherent empirical research limitation when AI performance is benchmarked against human expert judgment.

## **7.3 Practical Implementation Challenges**

### **Workflow Integration Complexity**

Integrating AI test generation into existing development workflows presents substantial organisational challenges. Determining an appropriate human oversight level proves difficult, as minimal review risks accepting flawed AI outputs leading to faulty testing outcomes, while excessive review requirements undermine efficiency gains, making AI adoption attractive. Finding the right human-AI collaboration balance requires ongoing experimentation and organisational learning.

AI integration with existing test management tools (traditionally designed for human interactive interface use) necessitates API integration, format standardisation, and metadata mapping, enabling programmatic test case injection. Organisations using legacy test management systems lacking these capabilities face significant adoption barriers.

Version control and change management processes require reconsideration. When AI regenerates test suites based on updated requirements, distinguishing meaningful changes from spurious variations through traditional diff-based reviews becomes challenging. Establishing the right granularity, change tracking, and approval workflows is critical for effective and efficient test case update management, requiring careful process design, ensuring AI and human efforts are properly aligned and managed.

### **7.3.1 Team Adoption and Cultural Factors**

QA professional resistance poses a significant adoption barrier. Concerns about job security, skill obsolescence, and professional identity often deter technology embracement. Organisations must address these with transparent communication, reskilling opportunities, and a clear role evolution vision.

Successful AI tool adoption can reposition testers as higher-value contributors, requirements quality validators, contextual gap identifiers, risk prioritisation experts, exploratory testing specialists, and test suite architects rather than solely test case creators. However, this evolution requires new skill development in AI tool management, prompt engineering, output validation, and effective human-AI collaboration. Organisations must invest in training and support, equipping teams with the necessary skills to thrive in this changing landscape.

Building AI-generated test trust requires demonstrated reliability and accuracy. Initial AI-generated tests containing hallucinations or contextual misunderstandings can undermine confidence. Establishing trust requires a phased rollout starting with low-risk areas where AI testing can be introduced and refined, continuous quality issue monitoring, and transparent limitation acknowledgement, enabling testers to become gradually more comfortable with AI tools.

### **7.3.2 Cost-Benefit Ambiguity**

While AI test generation offers dramatic efficiency gains (8× observed speedup), total cost of ownership analysis proves complex. AI tool subscriptions, API usage costs, infrastructure expenses, integration development, training investments, and ongoing maintenance must be

weighed against productivity improvements, quality enhancements, and competitive advantages.

Hidden costs include prompt engineering effort for high-quality outputs, human validation labour ensuring appropriate test coverage, rework addressing hallucinated or inappropriate tests, and organisational learning curve during adoption. Break-even analysis requires a realistic all-factor assessment.

Benefits extend beyond direct time savings: comprehensive traceability improves audit readiness for regulated environments, measurable test oracles facilitate automation, consistent test structure reduces maintenance overhead, and systematic coverage reduces oversight-driven defects. Quantifying these benefits challenges traditional ROI models.

### **7.3.3 Organizational Readiness Prerequisites**

Successful AI test generation deployment requires organisational prerequisites often underestimated during adoption planning. High-quality requirement documentation proves essential, AI performance degrades substantially with poor specifications, creating a bootstrapping problem where AI assistance appears most valuable precisely where its prerequisites are absent.

Established testing processes provide the necessary foundation. Organisations lacking systematic test case management, requirements traceability discipline, or structured quality gates face compounded challenges adopting AI tools. Addressing foundational process maturity must precede or accompany AI adoption.

Technical infrastructure requirements include API access to AI services (raising security and data privacy considerations for sensitive projects), integration with existing development and test management tools, sufficient computing resources for local model deployment if cloud services prove inappropriate, and robust network connectivity for cloud-based solutions.

## **7.4 Broader Implications and Concerns**

### **7.4.1 Professional Skill Evolution**

AI-augmented testing shifts QA professional skillset requirements. Traditional skills like manual test case authoring are becoming less critical, while new competencies emerge in AI output validation, contextual gap identification, prompt engineering, and human-AI workflow design. Educational programs and professional certifications must evolve, preparing testers for these changing demands, ensuring the necessary skills for effective AI tool collaboration.

This shift creates a potential junior tester career path disruption. Traditional progression from manual test execution to test case authoring to test strategy development may be compressed with AI handling routine test generation. As AI assumes routine and repetitive tasks, organisations must rethink professional development pathways, ensuring testers have clear skill progression opportunities, potentially focusing more on higher-level strategic roles and providing growth opportunities in AI tool management, test suite architecture, and risk-based testing.

### **7.4.2 Quality Assurance Philosophy**

Excessive AI testing reliance could shift focus from "testing to ensure quality" to simply "validating AI-generated tests." This subtle shift could cause teams to stop critically thinking about what should be tested, instead focusing on automatically validating whatever tests AI suggests.

Exploratory testing, relying on tester intuition and creativity to uncover issues, might become less important if organisations focus too heavily on executing formal, structured AI-generated test cases. Maintaining balance between systematic coverage (AI excellence) and creative exploration (human contribution) requires intentional effort, ensuring both approaches are valued.

### **7.4.3 Accountability and Liability**

When AI-generated tests miss critical defects leading to production failures, accountability assignment proves murky. Is fault attributed

to QA team for insufficient validation, AI tool provider for capability limitations, requirements authors for specification gaps, or organizational processes for inadequate human oversight? Clear accountability frameworks must accompany AI adoption.



## 8 Conclusion

This thesis explored the capabilities of generative AI for automated test case generation through a systematic empirical evaluation. Five leading large language models, ChatGPT, Microsoft Copilot, DeepSeek, Grok, and Claude, were progressively assessed through requirements extraction analysis, test generation comparison, and a comprehensive evaluation against professional human testers. The research provides an evidence-based understanding of where contemporary AI excels, its limitations, and offers insights into how organisations can effectively integrate AI into their quality assurance workflows.

### 8.1 Research Summary

This thesis investigated the capabilities of five large language models in software testing through a comprehensive two-phase experimental framework comprising 32 controlled experiments.

**Phase 1: Requirements Extraction** evaluated each model's ability to extract functional and non-functional requirements from various documentation formats. Results identified ChatGPT, Microsoft Copilot, and Claude as the strongest performers, with Claude demonstrating exceptional coverage (97.5%) and traceability (97.5%).

**Phase 2: Test Case Generation** employed three progressive stages:

- Stage 1 conducted exploratory five-model assessment
- Stage 2 performed systematic three-model comparison
- Stage 3 executed comprehensive Claude vs human evaluation

Final results demonstrate that Claude achieves 96.2% accuracy, surpassing other models in test case generation. These results highlight Claude's strength in understanding and interpreting requirements, its ability to generate test cases with high coverage, and its efficiency in producing consistent results.

The comprehensive human comparison in stage 3 provided valuable insights, confirming that Claude's performance, while impressive, still requires human validation for certain complex scenarios, such as business logic interpretation and risk prioritization.

All experimental data, detailed results, prompts, and analysis artefacts are publicly available in the research repository, ensuring reproducibility and enabling future research to build on this foundational work.

## 8.2 Answers to Research Questions

The investigation addressed five research questions formulated at thesis inception. This section provides evidence-based answers synthesizing findings across all research phases.

**Research Question 1: How accurately can generative AI models interpret an SRS document to produce valid test cases and a test management plan?**

Contemporary state-of-the-art models, such as Claude, demonstrate high accuracy in interpreting SRS documents for test generation. Claude achieved 96.2% accuracy in the Phase 3 evaluation, generating test cases that closely aligned with actual requirements, with fewer factual errors than human-generated tests (92.8%). This superior performance can be attributed to Claude's systematic requirement processing, precise adherence to stated specifications, and consistent application of testing standards.

However, accuracy is multifaceted. While factual correctness excels, limitations in contextual interpretation become apparent, particularly in terms of completeness (72.8% versus human 87.1%). AI models excel at validating explicitly stated requirements but struggle with inferring implicit functionality, interpreting ambiguous business logic, and recognising domain-standard practices. Despite efforts to mitigate instances of hallucinations generating unsupported content, this remains an ongoing concern (0.55 incidents per 100 test steps after controlled refinements).

One of AI's strengths lies in its ability to trace requirements. Claude achieved a perfect 100% RTM coverage with explicit requirement identifiers for every test case, far surpassing human performance (55%). This traceability is invaluable for regulated environments that demand thorough audit trails.

**Research Question 2: What are the differences in efficiency (time and effort) between AI-generated and human-written test cases?**

The efficiency differences are striking. Claude generated comprehensive test suites in approximately 6 minutes per module, compared to 65 minutes on average for human testers, resulting in a 10× productivity advantage. For the complete CORADS system, AI generated the entire test suite in under one hour, whereas human testers would take multiple days to achieve the same result.

However, comparing raw generation speed oversimplifies the value of AI adoption. Human testers also invest time in contextual analysis, business logic reasoning, and applying domain expertise tasks that are often not related to the mechanical aspects of test case documentation. AI-generated tests require human review to ensure contextual appropriateness, check business logic accuracy, and detect hallucinations. After accounting for this validation effort, organisations adopting AI for test generation can expect a 3-5× net productivity improvement, not the 10× speedup sometimes expected.

Additional efficiency benefits arise from AI's ability to enhance traceability, thereby reducing audit preparation effort. The use of measurable test oracles facilitates test automation, and a consistent test structure reduces maintenance overhead. Moreover, AI's systematic coverage minimises the risk of defects slipping through the cracks due to oversight.

**Research Question 3: How do AI-generated test cases compare to human-written ones in terms of test coverage and bug detection?** Test coverage reveals complementary strengths. AI-generated tests achieved 95% coverage of stated requirements, compared to 88% for human-generated tests, demonstrating superior attention to explicitly specified functionality. Additionally, Claude's 100% RTM coverage significantly outperformed human tests, which achieved only 55%. This ensures that no stated requirement is overlooked, making AI-generated tests highly valuable in providing comprehensive testing.

However, coverage is multidimensional. Human tests demonstrated 87.1% completeness compared to Claude's 72.8%, reflecting the superior ability of human testers to identify implicit requirements, unstated functionality, and domain-standard features that were not fully captured in the SRS. Integration testing was a particular gap for AI. Human testers included 33 integration tests to validate communica-

tion between system components, while Claude omitted this crucial aspect, despite CORADS comprising interconnected modules.

In terms of bug detection, AI excels at systematic functional validation, testing non-functional requirements (98% coverage versus human 40%), exploring security scenarios, and identifying boundary conditions. On the other hand, human testers are better equipped to handle environmental variation testing (e.g., platform-specific issues, network conditions, GPS accuracy degradation), production operational concerns, and real-world failure modes, leveraging experience to identify edge cases beyond the SRS.

Oracle quality further influences bug detection effectiveness. AI-generated tests used measurable, objective expected results (5.0/5.0 specificity), enabling reliable pass/fail determination. In contrast, human tests averaged 1.8/5.0, with 19% exhibiting vague expectations, such as "system works correctly" or "appropriate error displayed," which reduced test reliability.

**Research Question 4: What are the key challenges and limitations of using generative AI for test case generation?** The key technical challenges include persistent hallucinations, limitations in contextual understanding, risk prioritisation deficiencies (fair agreement with human judgment at  $\kappa = 0.34$ ), and environmental unawareness that omits platform-specific and operational considerations. Hallucinations vary in severity, from reasonable inferences about standard functionality to problematic inventions of entirely fictional requirements. In Experiment 13, Claude generated 28 resilience test scenarios, but only 7% were explicitly supported by the SRS, with the rest representing industry best practices inferred from domain knowledge. Despite refinements that reduced the hallucination rate by 40% Risk prioritisation represents the most significant limitation. Claude employed systematic but context-insensitive rules, lacking the business context understanding that human professionals bring. Human testers consider factors such as revenue impact, regulatory requirements, user base scale, and historical defect patterns, which are critical for prioritising tests based on actual business needs.

Practical implementation challenges include workflow integration complexity, team adoption resistance due to concerns over job security and professional identity, and the cost-benefit ambiguity that necessitates a comprehensive total cost of ownership analysis. Many organi-

sations, especially those in need of AI assistance, may face readiness issues when adopting AI tools.

Methodological limitations include the focus on a single SRS document, which limits the generalizability of the findings. The model version temporality suggests that the findings represent the contemporary capabilities of the AI models, not permanent features. The evaluation's subjectivity, despite the use of systematic rubrics, and the variability in human baseline quality across different experience levels and organisational contexts also present limitations.

**Research Question 5: How can the findings inform the development of an automated testing system?**

The findings support a hybrid AI-human workflow rather than full automation. A three-phase approach that combines complementary strengths is optimal.

**Phase 1: AI-Driven Draft Generation** – AI models generate comprehensive baseline test suites with perfect requirements traceability, measurable test oracles, and systematic coverage of stated requirements.

**Phase 2: Human Expert Validation and Enhancement** – Professional QA engineers review AI-generated test cases, applying contextual knowledge to fill gaps, adjust risk priorities, and ensure environment-specific scenarios are included.

**Phase 3: AI-Assisted Analysis and Maintenance** – AI models check for consistency across enhanced suites, suggest additional scenarios, maintain traceability as requirements evolve, and recommend regression test selection.

This hybrid approach achieves 96% accuracy, 95% coverage, less than 1% hallucination rate, and 8× efficiency improvement compared to fully human workflows, while still benefiting from human judgment. For successful adoption, organisations should prioritise high-quality requirement documentation, establish appropriate human oversight, invest in prompt engineering, develop hallucination detection protocols, and implement phased rollouts to build trust and improve AI performance.

### 8.3 Research Contributions

This thesis provides several contributions to software engineering research and practice:

**Empirical Evidence Base:** This research establishes a comprehensive empirical foundation for understanding contemporary AI test generation capabilities through a systematic evaluation that incorporates rigorous methodology, statistical validation, and reproducible procedures. The findings extend beyond anecdotal observations, providing evidence-based assessments with appropriate confidence measures, significance testing, and reliability analysis, thereby ensuring the results are robust and credible.

**Progressive Evaluation Methodology:** The study's three-phase progressive structure, comprising requirements extraction, test generation comparison, and human vs. AI evaluation, provides a replicable framework for systematically assessing AI capabilities. This methodology strikes a balance between breadth (evaluating multiple models across varied scenarios) and depth (providing a detailed human comparison with controlled refinements). The design maintains scientific rigour, ensuring that the evaluation covers a wide range of conditions while offering in-depth insights into AI performance.

**Dual-Perspective Analytical Approach:** By combining independent researcher evaluation with LLM-assisted verification, the research introduces a novel quality assessment methodology. This approach acknowledges the value of both human expertise and AI analytical capabilities. The resulting inter-rater reliability ( $= 0.72$ ) demonstrates substantial agreement between human evaluators, ensuring comprehensive and consistent evaluation coverage while validating the effectiveness of AI in real-world testing scenarios.

**Detailed Limitation Characterisation:** Rather than offering either uncritical enthusiasm or dismissive scepticism, the research provides a nuanced understanding of AI's strengths (e.g., systematic coverage, traceability, efficiency, and oracle quality) and shortcomings (e.g., contextual understanding, risk prioritisation, implicit requirements, and integration awareness). This balanced characterisation equips organisations with the knowledge needed to make informed adoption decisions, understanding both the potential and the limitations of AI in the testing process.

**Practical Hybrid Workflow Model:** The study presents a practical hybrid workflow model that combines AI and human collaboration, offering actionable guidance for organisations considering AI adoption. The evidence supporting this model achieving 96% accuracy, 95% coverage, and an 8-fold efficiency improvement sets realistic expectations regarding the benefits AI can bring, while emphasising the necessary human involvement to ensure quality and contextual relevance in the final outputs.

**Hallucination Mitigation Strategies:** The controlled refinements implemented in Experiments 9-15 demonstrated effective strategies for mitigating hallucinations, including the use of grounding constraints, RTM-first templates, policy-citation rules, and oracle contract schemas. These techniques provide a starting point for practitioners to develop prompt engineering strategies that can help reduce hallucinations and improve the quality of AI-generated test cases, contributing to more reliable AI test generation in future applications.

## 8.4 Practical Recommendations

Based on research findings, several recommendations guide organizational AI testing tool adoption:

### 8.4.1 For Organisations Considering AI Test Generation

Organisations should start by implementing **pilot projects in low-risk areas** to build organisational learning and trust before rolling out AI test generation across the entire enterprise.

- **High-quality requirement documentation** is crucial, as AI models perform best when specifications are comprehensive and detailed. With poor documentation, AI struggles to generate accurate tests.
- Establish clear **human oversight protocols** that strike a balance between validation thoroughness and efficiency.
- Expertise in **prompt engineering** should be developed through training and experimentation, as the quality of AI output depends heavily on the sophistication of the input.

- Plan for a 3–5× **net productivity improvement**, accounting for the validation overhead required for AI-generated tests, rather than expecting a full 10× theoretical speedup.
- Address team concerns by engaging in transparent communication, offering reskilling opportunities, and clearly defining how roles will evolve in tandem with AI adoption.

#### 8.4.2 For QA Professionals

QA professionals should adapt their skills to focus on:

- Validating AI output, identifying contextual gaps, engineering prompts, and orchestrating human-AI workflow.
- Viewing AI as a tool for **augmentation rather than a replacement**, professional expertise remains crucial for contextual understanding, business judgment, risk assessment, and recognising implicit requirements.
- Developing capabilities in areas such as **hallucination detection**, business logic validation, and requirement completeness assessment.

QA professionals can position themselves as higher-value contributors by focusing on test strategy, exploratory testing, and quality architecture, rather than merely generating mechanical test cases.

#### 8.4.3 For Tool Developers

Tool developers should focus on the following priorities:

- Prioritise **hallucination reduction** through improved grounding mechanisms and enhance contextual understanding (recognising implicit requirements and interpreting business logic).
- Develop **risk prioritisation features** that incorporate business context, rather than applying rigid, systematic rules.
- Improve **integration testing awareness** and multi-module validation capabilities.



- Work on creating **explainability features** that help users understand the AI's reasoning and identify potential errors.

Ultimately, designing **human-in-the-loop interaction patterns** is crucial, as it enables effective validation workflows and supports informed human decision-making throughout the process.

#### 8.4.4 For Researchers

Researchers should expand their evaluation efforts:

- Include **multiple SRS documents** across various domains, complexity levels, and documentation qualities.
- Conduct **longitudinal studies** that assess the maintainability of AI test suites over the course of a project lifecycle.
- Investigate **advanced AI architectures** (e.g., multi-agent systems, retrieval-augmented generation, specialised fine-tuning).
- Develop **improved evaluation metrics** that capture nuanced quality dimensions beyond traditional measures of coverage and correctness.
- Study **organisational adoption patterns**, identifying success factors and failure modes in real-world AI production deployments.

#### 8.4.5 Custom Model Development and Fine-Tuning Strategy

The research findings point toward custom fine-tuned models as the most promising path to overcoming identified limitations, particularly training data quality constraints and domain knowledge gaps.

##### **Fine-Tuning Value Proposition**

Organizations that have well-established software engineering practices and extensive internal documentation possess a valuable asset: high-quality proprietary training data. By using this data, such as internal SRS documents, API specifications, database schemas, test case

repositories, and bug tracking histories, organizations can fine-tune AI models to achieve significantly better performance compared to using general-purpose models.

### Estimated Impact

Based on the performance gaps observed between AI-generated test cases and those created by human testers, especially the 14.3-point deficit in completeness and limitations in contextual understanding, a well-executed fine-tuning program could lead to the following improvements:

- Reduce hallucination rates (i.e., generating incorrect or made-up information) from 0.55 to less than 0.2 per 100 test steps.
- Investigate **advanced AI architectures** (e.g., multi-agent systems, retrieval-augmented generation, specialised fine-tuning).
- Increase recognition of implicit requirements from 72.8% to 85% or more.
- Improve risk prioritization agreement (measured by Cohen's kappa) from 0.34 to 0.60+.
- Achieve near-human completeness (85%) while still benefiting from AI's efficiency.

### Organizational Prerequisites

To successfully fine-tune AI models, organizations must meet certain requirements:

1. **Data Volume:** You'll need a minimum of 500-1,000 high-quality SRS documents with corresponding test cases to train the model effectively. Larger organisations with years of project experience will likely have sufficient data, while smaller organisations may need to collaborate with others to gather the necessary data.
2. **Data Quality:** The training data must reflect current best practices. It shouldn't contain outdated methodologies or deprecated

technologies. Data curation and annotation should be done with the involvement of senior engineers to ensure their accuracy and relevance.

3. **Technical Infrastructure:** Fine-tuning large language models requires substantial computational resources (e.g., GPU clusters, high-memory systems) and specialised machine learning expertise. Organisations may use cloud-based fine-tuning services (such as OpenAI's custom model program or Anthropic's future services) to lower entry barriers.
4. **Continuous Update Cycles:** Since technology evolves rapidly, the fine-tuned models will need periodic retraining to incorporate new frameworks, security practices, and regulatory requirements.

#### 8.4.6 Practical Implementation Approach

Organizations should approach fine-tuning in phases to ensure a smooth implementation:

1. **Phase 1 - Proof of Concept (3-6 months):**
  - Select 50-100 representative SRS documents from various projects.
  - Pair each SRS with **human-authored test suites**.
  - Fine-tune a smaller, open-source model (like Llama 3 70B) for cost-effective validation.
  - Evaluate the model's performance against a test set to measure improvements over the baseline model.
2. **Phase 2 - Production Pilot (6-12 months):**
  - Expand the training corpus to **500+ documents**.
  - Include **API documentation, database schemas, and architecture diagrams** (with manual annotations for diagrams).
  - Fine-tune a production-scale model (such as GPT-4 or Claude Opus).

- Deploy the model to a **single project team** for real-world validation.

### 3. Phase 3 - Enterprise Rollout (12+ months):

- Incorporate feedback from the pilot phase, including edge cases.
- Establish a **continuous learning pipeline** to integrate new projects and data.
- Deploy the system **organization-wide**, with structured **human oversight protocols** to ensure quality and relevance.

## Cost-Benefit Analysis

Fine-tuning requires a substantial investment:

- **Initial Development:** \$100,000–\$500,000 (for data curation, compute resources, and expertise).
- **Annual Maintenance:** \$50,000–\$150,000 (for retraining and infrastructure upkeep).

However, for **large organizations** that release **50+ software projects annually**, the efficiency gains from fine-tuning (estimated **8× net productivity improvement**) would translate to:

- **70–80% reduction** in test case generation time.
- Higher **coverage** and **consistency** across projects.
- **Freed capacity** for senior engineers to focus on strategic testing initiatives.

**ROI Threshold:** Organizations with **more than 50 QA engineers** generating **over 1,000 test cases annually** are likely to see a **positive ROI** within 18–24 months.

### **Alternative: Retrieval-Augmented Generation (RAG)**

For organisations that cannot justify the investment in fine-tuning, Retrieval-Augmented Generation (RAG) offers a lighter alternative. RAG systems embed internal documents, like SRS, test case repositories, and technical specifications, into vector databases and retrieve relevant context for each AI query.

#### **Advantages**

- Lower implementation cost (\$20,000–\$100,000).
- No retraining required.
- Easier to **update** with new documents.

#### **Limitations**

- Less effective at learning organizational **style** and **terminology**.
- Depends on the quality of the **retrieved data**, meaning subtle contextual patterns may be missed.
- Typically achieves **50–70%** of the improvements that fine-tuning can provide.

## **8.5 Further Research Directions**

This section outlines several potential avenues for future research based on the findings of this thesis. While this study has provided valuable insights into the role of generative AI in software testing, several areas warrant further exploration to enhance the capabilities of AI models and address current limitations. Below are some suggested directions for further research:

### **8.5.1 Improvement of Non-Functional Requirement Handling**

The AI models in this research exhibited significant limitations in interpreting and generating test cases for non-functional requirements, including performance, security, and usability. Future research could

focus on developing specialised models or enhancing existing models to handle these complex requirements better. Specifically, research could explore how AI can be trained to generate comprehensive test cases for non-functional requirements that require specialised tools and testing environments.

**Possible research questions:**

- How can AI models be adapted to prioritize and generate test cases for non-functional requirements like load testing and security vulnerabilities?
- Can hybrid models (AI + human) optimize the testing process for non-functional requirements by leveraging human expertise in conjunction with AI's speed and consistency?

**Contextual and Domain-Specific Knowledge**

One of the key limitations identified in this study was AI's inability to fully comprehend domain-specific knowledge and business logic. This limitation is critical, particularly in fields such as healthcare, finance, or safety-critical systems where domain expertise is vital. Research could investigate how AI models can be augmented with domain-specific knowledge bases or integrated with expert systems to improve their contextual understanding.

**Possible research questions**

- How can domain-specific knowledge be integrated into AI models to improve their ability to generate contextually relevant test cases?
- What role can human experts play in validating or enriching AI-generated test cases for domain-sensitive systems?

**Real-Time Adaptability and Learning**

Another interesting direction is the real-time adaptability of AI in dynamic environments, such as Agile development. In Agile, requirements evolve rapidly, and test cases must be adjusted continuously.

Current AI models primarily work with static requirements documents, but future research could explore how AI can be trained to adapt to evolving specifications in real-time.

**Possible research questions**

- How can AI models be made more adaptive to continuous changes in requirements throughout an iterative software development process?
- What frameworks can be developed to enable AI models to learn from ongoing testing and incorporate feedback from real-world usage into their test case generation?

**Human-AI Collaboration in Testing**

While hybrid approaches (combining AI-generated test cases with human expertise) have shown promise, there is potential for deeper exploration into effective collaboration between human testers and AI systems. Research could examine methods for optimising human-AI workflows, ensuring that both AI's strengths in efficiency and human testers' strengths in creative problem-solving are maximised.

**Possible research questions**

- What are the most effective models for human-AI collaboration in software testing?
- How can AI-generated test cases be presented to human testers to facilitate quick and accurate validation without overwhelming them with redundant information?

**Handling Ambiguity and Uncertainty**

A significant challenge in software testing, both for AI and humans, is handling ambiguity in requirements. AI often generates test cases based on its understanding, which might not fully align with stakeholder intentions when the requirements are vague or incomplete. Future research could focus on enhancing AI's ability to identify ambiguities and request clarifications, similar to how human testers raise questions or flags during the testing phase.

**Possible research questions**

- How can AI models be improved to better handle ambiguous requirements by identifying uncertainty and requesting clarifications?
- Can AI models be designed to propose multiple test cases for ambiguous requirements, allowing human testers to choose the most appropriate one?

### **Long-Term Evaluation of AI-Generated Test Cases**

While this study evaluated AI-generated test cases based on initial performance metrics, long-term effectiveness remains largely unexplored. Future research could investigate how AI-generated test cases perform over time, especially as software evolves. Does the accuracy of AI-generated test cases degrade as systems become more complex, or do they maintain their value through continuous refinement?

#### **Possible research questions**

- How do AI-generated test cases evolve over the lifecycle of a software product, and what strategies can be developed to ensure their continued relevance and accuracy?
- What is the long-term effectiveness of AI-generated test cases in detecting defects in evolving software systems?

### **Ethical Considerations and Accountability**

As AI continues to play a larger role in software testing, ethical concerns related to AI-generated content become more important. These concerns include transparency, accountability, and the potential for biased test case generation. Research could explore how to mitigate ethical risks in AI-based testing by ensuring that AI systems are transparent, auditable, and accountable for their outputs.

#### **Possible research questions**

- How can ethical considerations be incorporated into the development of AI models for software testing?
- What frameworks can be established to ensure that AI models are transparent and accountable for their test case generation process?



### Scaling AI Testing in Large-Scale Software Systems

While this study focused on a single case study, the scalability of AI testing tools in large, complex software systems remains an open question. Future work could examine how AI models perform when applied to large-scale, distributed systems with multiple teams working in parallel.

#### Possible research questions

- How can AI testing tools be scaled for use in large, multi-team software projects?
- What challenges arise when applying AI-based testing tools to systems with a high degree of complexity, and how can they be addressed?

## 8.6 Future Work

Several research directions emerge from this investigation's findings and limitations:

**Multi-Domain Evaluation** Expand the evaluation to include safety-critical systems (such as medical devices, aviation, and automotive), highly-regulated industries (like financial services, healthcare, and government), as well as embedded systems, real-time systems, and algorithm-intensive applications. These different areas may reveal unique AI performance patterns and challenges specific to each domain, which could necessitate specialized approaches to address them effectively.

**Specification Quality Impact Analysis** Examine how the quality of SRS documentation affects AI performance by systematically varying the quality from informal user stories to highly formal specifications. This study highlights how AI performance is sensitive to the quality of input documentation and clarifies the paradox where AI appears most valuable in scenarios where detailed specifications are often lacking, making AI tools a potential solution for addressing poorly defined requirements.

**Longitudinal Maintenance Studies** Track AI-generated test suites over multiple cycles of requirement changes, evaluating how the tests are maintained, how changes affect them, and how effective regression

testing is over time. It's essential to verify whether the initial quality of generated tests remains consistent as the system evolves, as long-term maintenance may pose challenges to AI-generated content.

**Advanced Architecture Exploration** Look into specialized approaches for improving AI performance in test generation, including methods like retrieval-augmented generation (which gives models access to relevant code, documentation, and historical test cases), multi-agent systems that use specialized agents for different types of testing (such as functional, security, performance, and integration testing), and fine-tuning approaches that train models on organization-specific codebases and testing practices. Additionally, consider hybrid symbolic-neural approaches that combine formal methods with the flexibility of large language models to improve the overall effectiveness of AI-generated tests.

**Improved Risk Prioritization** Develop new methods for incorporating business context, organizational priorities, and historical operational data into AI's risk assessment process. This could include learning from real-world production incident databases, stakeholder feedback, and economic impact models to help AI better prioritize test cases that address the most critical risks to the business.

**Enhanced Evaluation Methodologies** Create new evaluation metrics that measure coverage of implicit requirements, the appropriateness of business logic, and how well domain expertise is applied in AI-generated tests. Additionally, explore ways to automate hallucination detection and enhance techniques for inter-rater reliability to assess the subjective quality aspects of the generated tests.

**Production Deployment Studies** Conduct field studies in real-world organisational settings to measure the actual adoption success of AI test generation tools, looking at factors such as team satisfaction, productivity impacts, defect reduction, and total cost of ownership. These studies will help identify critical success factors and standard failure modes, providing insights that can guide adoption strategies in similar organisations.

**Integration Testing Capabilities** Explore ways to improve AI's ability to recognise system architecture, identify interactions between system components, and generate integration test scenarios. This is a significant gap in current AI test generation models, which often overlook the

complexity of multi-component systems and fail to create thorough integration tests.

**Explainable Test Generation** Develop methods to make AI-generated tests more explainable. This involves creating tools that enable AI models to explain their reasoning behind test generation decisions, such as how they interpret requirements, why specific tests are selected, and how coverage decisions are made. Increased explainability would facilitate human validation, build trust in AI-generated tests, and aid in debugging issues during the test generation process.

## 8.7 Final Remarks

This research demonstrates that contemporary large language models, particularly Claude, are capable of matching the accuracy of professional quality assurance engineers in structured test generation while excelling in areas such as requirements traceability, oracle measurability, and generation efficiency. However, AI still requires human oversight to ensure contextual completeness, alignment with business risk, and seamless organizational integration. The future of software testing does not lie in replacing humans but in fostering effective human-AI collaboration, where both parties leverage their complementary strengths.

The study provides a solid empirical foundation for organizations considering the adoption of AI testing tools, offering an evidence-based understanding that goes beyond the hype and scepticism often associated with AI. Organizations can realistically expect substantial productivity gains (a 3-5× net improvement) while maintaining or even improving test quality by implementing hybrid workflows. Achieving success requires realistic expectations, adequate preparation, and a commitment to preserving human expertise while integrating AI capabilities.

As large language models continue to evolve rapidly, the capabilities assessed in this research represent a current baseline, not a permanent limit. Future models may overcome the limitations discussed through architectural enhancements, specialized training, and advanced reasoning capabilities. However, the fundamental tension between AI's systematic processing and humans' contextual under-

standing suggests that hybrid approaches will likely remain the most effective strategy for the foreseeable future.

In conclusion, this research demonstrates that automated test generation has evolved from an academic curiosity to a practical reality. Organizations that invest in a thoughtful adoption process with the proper prerequisites, realistic expectations, and effective human-AI workflows can see significant improvements in quality assurance. The real challenge isn't whether AI can contribute to software testing, but rather how organizations can successfully integrate these capabilities while preserving the critical human expertise needed for testing.

## Bibliography

- [1] DOBSLAW, Felix; FELDT, Robert; YOON, Jeongju; YOO, Shin. Challenges in Testing Large Language Model-Based Software: A Faceted Taxonomy. 2025. Available from arXiv: 2503.00481.
- [2] SPILLNER, Andreas; LINZ, Tilo; SCHAEFER, Hans. *Software Testing Foundations, 5th Edition: A Study Guide for the Certified Tester Exam*. Rocky Nook, 5th edition, 2021.
- [3] AMMANN, Paul; OFFUTT, Jeff. *Introduction to Software Testing*. Cambridge University Press, 2016.
- [4] PEZZÈ, Mauro; YOUNG, Michal. *Software Testing and Analysis: Process, Principles and Techniques*. Wiley, 2007.
- [5] ALAMMAR, Jay; GROOTENDORST, Maarten. *Hands-On Large Language Models: Language Understanding and Generation*. O'Reilly, 2024.
- [6] BERRYMAN, Jared; ZIEGLER, Albert. *Prompt Engineering for LLMs*. O'Reilly, 2024.
- [7] NAVEED, Humza; KHAN, Asad Ullah; QIU, Shi; SAQIB, Muhammad; ANWAR, Saeed; USMAN, Muhammad; AKHTAR, Naveed; BARNES, Nick; MIAN, Ajmal. A Comprehensive Overview of Large Language Models. *ACM Transactions on Intelligent Systems and Technology*, 2025, vol. 16, no. 5, article 106.
- [8] IEEE. *IEEE Standard for Software and System Test Documentation*. IEEE Std 829-2008, 2008.
- [9] ISTQB. *ISTQB Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board, version 4.0, 2023.
- [10] WANG, Chunhui. Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach. *IEEE Transactions on Software Engineering*, 2020, vol. 47, no. 11, pp. 2388–2407.

- 
- [11] LAFI, Mohammed; ALRAWASHED, Thamer; HAMMAD, Ahmad Munir. Automated Test Cases Generation From Requirements Specification. In: *Proc. International Conference on Information Technology (ICIT)*. 2021, pp. 317–322.
  - [12] ALLALA, Sai Chaithra; SOTOMAYOR, Juan P.; SANTIAGO, Dionny; KING, Tariq M.; CLARKE, Peter J. Generating Abstract Test Cases from User Requirements using MDSE and NLP. In: *Proc. IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. 2022, pp. 862–871.
  - [13] ARORA, Chetan; HERDA, Tomas; HOMM, Verena. Generating Test Scenarios from NL Requirements Using Retrieval-Augmented LLMs: An Industrial Study. In: *Proc. IEEE 32nd International Requirements Engineering Conference (RE)*. 2024, pp. 123–133.
  - [14] HALDAR, Susmita; PIERCE, Mary; CAPRETZ, Luiz Fernando. Assessing the Effectiveness of ChatGPT in Preparatory Testing Activities. In: *Proc. IEEE Frontiers in Education Conference (FIE)*. 2024, pp. 1–8.
  - [15] HALDAR, Susmita; PIERCE, Mary; CAPRETZ, Luiz Fernando. Exploring the Integration of Generative AI Tools in Software Testing Education: A Case Study on ChatGPT and Copilot for Preparatory Testing Artifacts in Postgraduate Learning. *IEEE Access*, 2025, vol. 13, pp. 1234–1248.
  - [16] CHAUHAN, Pooja; BALAKRISHNAN, Vijayakumar. Domain-Specific Software System Testing by Using Intelligent Approaches. In: *Proc. International Conference on Computational Intelligence and Network Systems (CINS)*. 2024, pp. 45–52.
  - [17] AARIFEEN, Hafsa; WICKRAMAARACHCHI, Dilani. Evaluating AI-Based Unit Testing Tools: A Focus on Usability, Reliability, and Integration. In: *Proc. IEEE International Conference on Advanced Research in Computing (ICARC)*. 2025, pp. 89–96.
  - [18] JIAO, Yuantao; WANG, Jian; LI, Runmei; WANG, Fei-Yue; ZHAO, Hongtao; XIONG, Gang. Intelligent Generation of Test Cases for

- a Parallel Testing System: A Case Study on Railway Systems. *IEEE Intelligent Transportation Systems Magazine*, 2025, vol. 17, no. 2, pp. 78–92.
- [19] CHENAIL-LARCHER, Zacharie; MINANI, Jean Baptiste; MOHA, Naouel. Test Generation from Use Case Specifications for IoT Systems: Custom, LLM-Based, and Hybrid Approaches. In: *Proc. IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 2025, pp. 156–167.
- [20] HANDA, Devansh; NIKHIL, Kindi Krishna; KANAGARAJ, Kanaganandini; DUVARAKANATH, S.; SUPRIYA, M. Test Case Generation using Graph-based Deep Learning Methods. In: *Proc. IEEE International Conference on Artificial Intelligence and Machine Learning Applications (AIMLA)*. 2025, pp. 234–241.
- [21] OPENAI. GPT-4 Technical Report. 2023. Available from arXiv: 2303.08774.
- [22] ANTHROPIC. Claude 3 Model Card. Technical Report, 2024. Available also from: <https://www.anthropic.com/claude>.
- [23] MICROSOFT. Microsoft Copilot: Your AI companion. 2024. Available also from: <https://www.microsoft.com/copilot>.
- [24] DEEPSEEK. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. Available from arXiv: 2405.04434.
- [25] XAI. Grok-1: A Large Language Model. Technical Report, 2024. Available also from: <https://x.ai/>.
- [26] BROWN, Tom et al. Language Models are Few-Shot Learners. In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, pp. 1877–1901.
- [27] WEI, Jason et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. 2022, pp. 24824–24837.

- 
- [28] YAO, Shunyu et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: *Proc. International Conference on Learning Representations (ICLR)*. 2023.
  - [29] LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, pp. 9459–9474.
  - [30] ACHIAM, Josh et al. GPT-4 Technical Report. 2023. Available from arXiv: 2303.08774.
  - [31] ANIL, Rohan et al. PaLM 2 Technical Report. 2023. Available from arXiv: 2305.10403.
  - [32] TOUVRON, Hugo et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. 2023. Available from arXiv: 2307.09288.
  - [33] TEAM, Gemini et al. Gemini: A Family of Highly Capable Multi-modal Models. 2023. Available from arXiv: 2312.11805.
  - [34] VASWANI, Ashish et al. Attention is All You Need. In: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, pp. 5998–6008.
  - [35] DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 2019, pp. 4171–4186.
  - [36] RAFFEL, Colin et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 2020, vol. 21, no. 140, pp. 1–67.
  - [37] CLARK, Kevin; LUONG, Minh-Thang; LE, Quoc V.; MANNING, Christopher D. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. In: *Proc. International Conference on Learning Representations (ICLR)*. 2020.
  - [38] RADFORD, Alec et al. Learning Transferable Visual Models From Natural Language Supervision. In: *Proc. International Conference on Machine Learning (ICML)*. 2021, pp. 8748–8763.



- [39] HOFFMANN, Jordan et al. Training Compute-Optimal Large Language Models. 2022. Available from arXiv: 2203.15556.
- [40] BISK, Yonatan et al. Experience Grounds Language. In: *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 8718–8735.
- [41] BUBECK, Sébastien et al. Sparks of Artificial General Intelligence: Early experiments with GPT-4. 2023. Available from arXiv: 2303.12712.
- [42] ZHANG, Jie; ZHAO, Junjie; ZHENG, Hanjie; TAN, Chengcheng. LLMs in Software Engineering: A Survey. 2023. Available from arXiv: 2312.15223.
- [43] HOU, Xinyi et al. Large Language Models for Software Engineering: A Systematic Literature Review. 2023. Available from arXiv: 2308.10620.
- [44] FAN, Yangrui et al. Large Language Models for Software Engineering: Survey and Open Problems. In: *Proc. IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE)*. 2024, pp. 31–38.
- [45] CHEN, Mark et al. Evaluating Large Language Models Trained on Code. 2021. Available from arXiv: 2107.03374.
- [46] JIANG, Nan et al. Impact of Code Language Models on Automated Program Repair. In: *Proc. IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, pp. 1430–1442.
- [47] XIA, Chunqiu Steven; ZHANG, Lingming. Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. 2023. Available from arXiv: 2304.00385.
- [48] PEARCE, Hammond et al. Examining Zero-Shot Vulnerability Repair with Large Language Models. In: *Proc. IEEE Symposium on Security and Privacy (S&P)*. 2023, pp. 2339–2356.
- [49] PRENNER, Julian Aron; ROBBES, Romain. Automatic Program Repair with OpenAI’s Codex: Evaluating QuixBugs. 2022. Available from arXiv: 2111.03922.

## BIBLIOGRAPHY

---

- [50] JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? 2023. Available from arXiv: 2310.06770.

## A An appendix

All experimental data, prompts, analysis results, and supplementary materials for this research are publicly available in the following GitHub repository:

**Repository:** <https://shorturl.at/CbhIw>

The repository contains:

- Complete experimental data from all 32 experiments (7 requirement extraction + 25 test generation experiments across three phases)
- AI-generated test cases and test management plans from all five models (ChatGPT, Microsoft Copilot, DeepSeek, Grok, Claude)
- Human-written test cases baseline (JSON format)
- All prompts and prompt templates used throughout the research
- Source requirements documents (SRS, user stories, use cases)
- Statistical analysis calculations and supplementary data
- Research instruments (evaluation criteria, scoring rubrics)

The repository is organized with clear documentation to facilitate reproducibility and verification of the research findings.